

Side-Channel and Fault Injection Attacks on VOLEitH Signature Schemes: A Case Study of Masked FAEST

Sönke Jendral and Elena Dubrova

KTH Royal Institute of Technology, Stockholm, Sweden
{jendral,dubrova}@kth.se

Abstract. Ongoing efforts to transition to post-quantum public-key cryptosystems have created the need for algorithms with a variety of performance characteristics and security assumptions. Among the candidates in NIST’s post-quantum standardisation process for additional digital signatures is FAEST, a Vector Oblivious Linear Evaluation in-the-Head (VOLEitH)-based scheme, whose security relies on the one-wayness of the Advanced Encryption Standard (AES). The VOLEitH paradigm enables competitive performance and signature sizes under conservative security assumptions. However, since it was introduced recently, in 2023, its resistance to physical attacks has not yet been analysed. In this paper, we present the first security analysis of VOLEitH-based signature schemes in the context of side-channel and fault injection attacks. We demonstrate four practical attacks on a masked implementation of FAEST in ARM Cortex-M4 capable of recovering the full secret key with high probability (greater than 0.87) from a single signature. These attacks exploit vulnerabilities of components specific to VOLEitH schemes and FAEST, such as the parallel all-but-one vector commitments, the VOLE generation, and the AES proof generation. Finally, we propose countermeasures to mitigate these attacks and enhance the physical security of VOLEitH-based signature schemes.

Keywords: Side-channel analysis · Fault injection · FAEST · Vector Oblivious Linear Evaluation in-the-Head (VOLEitH) · Post-quantum digital signature · Key recovery attack

1 Introduction

In 2024, the National Institute of Standards and Technology (NIST) published the first set of standards from its main competition for post-quantum cryptographic (PQC) algorithms for key-encapsulation mechanism ML-KEM [49] and digital signature schemes ML-DSA [48] and SLH-DSA [51]. In an effort to enable the use of PQC algorithms in a wider range of use cases, NIST launched a second competition, focusing on additional digital signature schemes based on different underlying problems and with different performance characteristics than ML-DSA and SLH-DSA [47].

Among the submissions selected by NIST [50] for the second round in this competition is FAEST, a digital signature scheme based on the Vector Oblivious Evaluation in-the-Head (VOLEitH) paradigm introduced by Baum et al. [16] in 2023. FAEST is designed to be existentially unforgeable under chosen message attacks (EUF-CMA) in the (quantum) random oracle model [17]. EUF-CMA security means that an adversary with access to the public key and a signing oracle cannot generate a valid signature for a new message.

The idea behind VOLEitH signature schemes is to use two-party VOLE correlations to facilitate a zero-knowledge proof of knowledge (ZKPoK) of the computation of a one-way function (OWF). The benefit of this approach is that, assuming the correctness of the ZKPoK, the security of the signature scheme relies primarily on the one-wayness of the OWF. In the case of FAEST, the OWF is the Advanced Encryption Standard (AES) and the ZKPoK proves knowledge of the AES state for each encryption round under a public plaintext and secret key, thus providing strong security guarantees based only on symmetric primitives, while offering competitive signing and verification performance and signature sizes.

However, the VOLEitH paradigm was introduced recently and FAEST is the only first-round submission in the NIST competition for additional digital signatures making use of this paradigm. Thus the security analysis of FAEST and the VOLEitH paradigm has not received as much attention as submissions based on more mature approaches. Previous attacks on PQC algorithms [5,23,42,9,10,44] have shown that even if algorithms are theoretically secure, their implementations might be vulnerable to physical attacks. To allow developers to design and implement effective countermeasures to such attacks, it is therefore important to identify the types of physical attacks that can be conducted on FAEST.

After the completion of the experiments presented in this paper, NIST announced the changes that submitters made for the second round of the competition. With these changes, three of the five MPCitH-based signature schemes (Mirath [1], MQOM [18], RYDE [6]) now offer a variant that makes use of the VOLEitH paradigm, while a fourth (formerly) MPCitH-based scheme, SDitH [2], now becomes the second VOLEitH-based scheme in the competition alongside the updated version of FAEST [14]. The presented analysis is therefore of even greater importance. In this paper, we analyse only the first-round variant of FAEST, but all our results still hold for its second-round variant, as well as for other VOLEitH-based schemes, as discussed in Section 12.

Contributions: In this paper, we present the first physical security analysis of VOLEitH-based signature schemes. The main contributions are:

- i) We provide a high-level evaluation of the susceptibility of VOLEitH-based signature schemes to side-channel and fault injection attacks, including the analysis of the underlying VOLE correlations and all-but-one vector commitments that establish these correlations.

- ii) We present two single-trace deep learning-assisted power-based side-channel attacks and two single-execution first-order voltage fault injection attacks on the masked implementation of FAEST by Degn, Eilath, and Nielsen [25].
- iii) We validate the feasibility of the attacks on an ARM Cortex-M4 processor and empirically evaluate their success rate.
- iv) We propose countermeasures to mitigate these attacks.
- v) We discuss the applicability of the attacks to the second-round variants of FAEST, SDitH, Mirath, RYDE, MQOM, PERK, LESS, and CROSS.

The presented physical security analysis helps identify key vulnerable components in VOLEitH-based signature schemes. Our experimental results show that, for side-channel attacks, such vulnerable components are operations involving witness bits and VOLE tags, while for fault injection attacks, they are operations related to the computation of VOLE and vector commitments. These results are expected to offer guidance to implementers of VOLEitH-based signature schemes and designers of countermeasures, helping them focus their efforts in the right direction.

It is also worth noting that, as the FAEST implementation [25] is still under active development, not all its components are protected by masking yet. To ensure the long-term relevance of the presented analysis, our side-channel attacks target components which are already protected by masking.

Organisation of the paper: The rest of this paper is organised as follows. Section 2 describes previous work. Section 3 provides background information on the VOLEitH paradigm and the FAEST algorithm. Section 4 outlines the experimental setup. Section 5 discusses the susceptibility of VOLEitH-based signature algorithms to physical attacks. Sections 6 and 7 present the side-channel attacks. Sections 8 and 9 present the fault injection attacks. Section 10 summarises the experimental results. Section 11 discusses potential countermeasures against the attacks. Section 12 discusses the applicability of the attacks to the second-round variants. Section 13 concludes the paper.

2 Previous work

To the best of our knowledge, the presented work is the first to evaluate the resistance of FAEST (and the underlying VOLEitH paradigm) to physical attacks, such as side-channel analysis and fault injection. Previous work has focused on the related Multiparty Computation in the Head (MPCitH) approach [37], which differs from VOLEitH (and FAEST) in a number of aspects. This section provides an overview of previous attacks and highlights the differences to the attacks presented in this paper.

Godard et al. [32] describe a single-trace side-channel attack on the reference implementation of the Syndrome Decoding in the Head (SDitH) signature scheme [3]. Specifically, they target the Galois field multiplications during the evaluation of the shares of a secret polynomial S for each of the parties and

apply a templated soft-analytical side-channel attack. They practically evaluate their attack on a STM32F407 (ARM Cortex-M4), and show that a number of coefficients of S can be extracted and used to recover the secret key with an enumeration complexity of $70\text{--}70^2$ for all security levels from a single trace. As a countermeasure, they propose the use of shuffling.

It is unclear how their attack would translate to FAEST, as the finite field multiplications in FAEST are mainly performed in the larger finite field $\mathbb{F}_{2^{128}}$, rather than $\mathbb{F}_{2^8}$ as in SDitH, and thus use a different multiplication algorithm. Note that, while Feneuil and Rivain [29] suggest that the VOLEitH paradigm can be seen as a special case of a variant of the Threshold Computation in-the-Head (TCitH) paradigm with GGM trees, the variant of SDitH that Godard et al. consider uses a different variant of TCitH that does not have such a relation to VOLEitH. The attack they present focuses on the underlying OWF, rather than the broader paradigm.

Gellersen et al. [31] describe two differential power-based side-channel attacks on Picnic [22]. Both attacks involve the LowMC block cipher [4], which is used as the OWF in Picnic. The first attack exploits the secret sharing prior to the multiparty LowMC computation, which leaks the Hamming weight of the unopened key shares. The second attack targets the computation of the S-boxes, which similarly reveals information about the unopened share. They practically evaluate their attacks on a NXP MK66FN2M0VMD18 (ARM Cortex-M4) and demonstrate that it is possible to recover the full secret key from fewer than 1000 traces, obtainable from fewer than 30 signatures. Seker et al. [54] propose a modified variant of Picnic that uses several masking gadgets to prevent these types of attacks. As FAEST is not based on multiparty computation, and uses AES instead of LowMC as the underlying OWF primitive, neither the attacks [31,54] nor the countermeasures are directly applicable.

Aranha et al. [8] also describe two side-channel attacks against Picnic. The first attack applies a similar idea to the previous attacks to the Picnic variant with preprocessing (Picnic3). They observe that information from an offline preprocessing step can be combined with side-channel information about a masked witness wire value to recover a secret key bit. They experimentally validate this attack on a STM32F407G-DISC1 (ARM Cortex-M4) and show that leakage becomes clear after 2,725 traces (and thus signatures). The second attack is an extension of the previous attacks [31,54] to Picnic3. They also propose a masking scheme that can prevent their attacks. Again, the attacks and countermeasures are not directly applicable to FAEST, due to differences in the underlying paradigms. However, the masked implementation of SHAKE which they present is also used in the masked implementation of FAEST [25] targeted in this paper, though none of our attacks focus on this component.

3 Background

3.1 Notation

In this paper, we follow the notation of [17]. Intervals are denoted as $[a..b] = \{a, \dots, b-1, b\}$ and $[a..b) = \{a, \dots, b-1\}$, vectors in bold font $\mathbf{x}$ and matrices in capitals $\mathbf{X}$. Indexing is denoted as $\mathbf{x}[i]$ for the i -th element of a vector, $\mathbf{X}|_i$ for the i -th row of a matrix, while $a \parallel b$ denotes the concatenation of a and b , and $\mathbf{x}_{[a..b]}$ the elements at indices a through b of a vector or list. Additionally, we use the notation $x[j]$ to refer to the j -th bit of a field element x , and $x \nmid y$ to denote that y is not divisible by x .

3.2 VOLEitH signature schemes

The VOLEitH paradigm for constructing secure digital signature schemes from VOLE-based ZKPoKs was first introduced by Baum et al. [16]. This section provides an overview of the main components of the paradigm, primarily based on the FAEST specification [17].

Fundamentally, the VOLEitH approach involves a number of VOLE correlations

$$q_i = \Delta \cdot u_i - v_i \quad \text{with } i \in [0..l) \quad (1)$$

over a finite field $\mathbb{F}_{2^k}$, where $q_i \in \mathbb{F}_{2^k}$ is a public *VOLE key*, $u_i \in \mathbb{F}_2$ is a secret random bit, $\Delta \in \mathbb{F}_{2^k}$ is the public *global key*, and $v_i \in \mathbb{F}_{2^k}$ is a secret *VOLE tag*.

These correlations are computed using all-but-one vector commitments based on a construction by Goldreich, Goldwasser, and Micali [33], which is referred to as a GGM tree. In the construction, a length-doubling pseudorandom generator (PRG) is used to compute a tree with pseudorandom seeds $(\mathbf{sd}_i)_{i \in [0..N)}$ as leaves, such that all-but-one of these seeds can be efficiently revealed (i.e. with logarithmic communication cost, by revealing full subtrees instead of individual leaves) through opening the vector commitment. From the seeds, a set of random bit strings $(\mathbf{r}_i)_{i \in [0..N)}$ is derived, again using a PRG. This allows computing a vector $\mathbf{u} \in \mathbb{F}_2^l$ of random bits and a vector $\mathbf{v} \in \mathbb{F}_{2^k}^l$ of VOLE tags as

$$\mathbf{u} = \sum_{i=0 \in \mathbb{F}_{2^k}}^{N-1} \mathbf{r}_i, \quad \mathbf{v} = \sum_{i=0 \in \mathbb{F}_{2^k}}^{N-1} i \cdot \mathbf{r}_i$$

and, by revealing seeds $\mathbf{sd}_i$ for all $i \neq \Delta$ for random global key $\Delta \in \mathbb{F}_{2^k}$, allows computing a vector $\mathbf{q} \in \mathbb{F}_{2^k}^l$ of VOLE keys as

$$\begin{aligned} \mathbf{q} &= \sum_{i=0 \in \mathbb{F}_{2^k}}^{N-1} (\Delta - i) \cdot \mathbf{r}_i \\ &= \Delta \cdot \sum_{i=0 \in \mathbb{F}_{2^k}}^{N-1} \mathbf{r}_i - \sum_{i=0 \in \mathbb{F}_{2^k}}^{N-1} i \cdot \mathbf{r}_i \\ &= \Delta \cdot \mathbf{u} - \mathbf{v} \end{aligned}$$

for the final correlation.

A useful property of the VOLE correlations is that they are linearly homomorphic, thus it is possible to perform arithmetic on them. For example, given two VOLE correlations $(\Delta, q_0), (u_0, v_0)$ and $(\Delta, q_1), (u_1, v_1)$ for a global key Δ , their sum can be represented by the VOLE correlation $(\Delta, q_0 + q_1), (u_0 + u_1, v_0 + v_1)$. This approach can also be extended to constant values and other linear operations, see [17].

To be able to use such VOLE correlations to construct a proof with specific witness bit values $\mathbf{w} \in \mathbb{F}_2^l$ instead of the random bits $\mathbf{u}$, the prover can compute a vector $\mathbf{d} := \mathbf{w} - \mathbf{u}$, such that

$$\mathbf{q}' := \mathbf{q} + \Delta \cdot \mathbf{d} = \Delta \cdot (\mathbf{u} + \mathbf{d}) - \mathbf{v} = \Delta \cdot \mathbf{w} - \mathbf{v} \quad (2)$$

can be used to form valid VOLE correlations for $\mathbf{w}$ instead [17].

As a simplification, the FAEST specification [17] and a number of algorithms in this paper occasionally interpret the vectors of the form $\mathbb{F}_{2^k}^l$ given in Eq. 2 instead as matrices of the form $\{0, 1\}^{l \times k}$. This yields the notions of a VOLE key matrix $\mathbf{Q}$ and a VOLE tag matrix $\mathbf{V}$, where each row corresponds to a single correlation of the form in Eq. 1, i.e. where the columns correspond to the bit decomposition of the field element. Clearly both representations are interchangeable.

To simplify the explanations given in later parts of this paper, we use this matrix representation to introduce the notion of active columns. Specifically, we say column i of the VOLE tag or key matrix is *active* if the i -th bit of the global key Δ , $\Delta[i] = 1$. From Eq. 2, it follows that the structure of the VOLE correlations is such that the active columns are the only columns where the VOLE key contains information about the witness bit. This notion is useful when attempting to recover information about the witness from the VOLE tag and key matrices.

As a further optimisation, Baum et al. [16] extend a method described for the SoftSpokenOT protocol of Roy [53] that allows constructing a VOLE for a larger field from multiple VOLEs over smaller fields. As the precise details of this construction are not relevant here, we instead refer to [16].

In practical terms, this optimisation means that τ individual VOLEs over smaller fields $\mathbb{F}_{2^{k_0}}$ and $\mathbb{F}_{2^{k_1}}$ are combined together to form a VOLE over the larger field $\mathbb{F}_{2^k}$. This is only possible if all τ VOLEs correspond to the same random bits $\mathbf{u}$, so prior to the adjustment for the witness bit values in Eq. 2, a number of correction values $\mathbf{c}_i = \mathbf{u}_i - \mathbf{u}_0$ are computed for $i \in [1.. \tau)$ by the prover (and later applied by the verifier in the same way as before), to ensure that all VOLE correlations use the same set of random bits (i.e. the random bits $\mathbf{u}_0$ computed for the first VOLE correlation). Additionally, this requires a consistency check to ensure that the prover does not cheat when computing the correction values, which we again do not show here and which can instead be found in the specification [17].

Based on the VOLE correlations, Baum et al. [16] then use the interactive QuickSilver ZKPoK protocol [59] to prove an arbitrary arithmetic circuit. Concretely, the specification [17] describes the computation of a linear gate a, b, c

as $w_\gamma := a \cdot w_\alpha + b \cdot w_\beta + c$ and $v_\gamma := a \cdot v_\alpha + b \cdot v_\beta$ by the prover and $q_\gamma := a \cdot q_\alpha + b \cdot q_\beta + c \cdot \Delta$ by the verifier, which does not require any communication. Further, it describes the computation of a multiplication gate a, b, c as $w_\gamma := w_\alpha \cdot w_\beta$ and $d_\gamma = w_\gamma - u_i$ by the prover (i.e. the prover commits to the output bit w_γ), and $q_\gamma := q_\gamma + d_\gamma \cdot \Delta$ by the verifier (i.e. the verifier verifies the validity of the output bit). The verifier is then able to check the multiplication by computing and checking

$$\begin{aligned} b_\gamma &:= q_\alpha \cdot q_\beta - q_\gamma \cdot \Delta \\ &= v_\alpha \cdot v_\beta + (w_\alpha \cdot v_\beta + w_\beta \cdot v_\alpha - v_\gamma) \cdot \Delta + \underbrace{(w_\alpha \cdot w_\beta - w_\gamma) \cdot \Delta^2}_{\text{Vanishes if prover is honest}} \\ &\stackrel{!}{=} a_0 + a_1 \cdot \Delta \end{aligned} \quad (3)$$

using values $a_0 := v_\alpha \cdot v_\beta$ and $a_1 := w_\alpha \cdot v_\beta + w_\beta \cdot v_\alpha - v_\gamma$ computed by the prover. Note that a_0 and a_1 are not revealed to the verifier directly, but are instead combined across all multiplication gates and then masked using another VOLE, as described in [17].

Using this approach, it is possible to prove an arbitrary arithmetic circuit corresponding to the evaluation of a OWF. A potential problem for using such a construction in the context of a signature scheme is that it provides a designated verifier proof, i.e. a proof which requires interaction between the prover (signer) and verifier to establish the VOLE correlations from the all-but-one vector commitments [16]. The central insight by Baum et al. [16] is that it is not necessary that the prover is unable to learn the global key Δ that the verifier chooses. Instead, it is sufficient to ensure that the prover is committed to their proof prior to learning the value of Δ , as this still prevents the prover from cheating in the proof. Then, by applying a Fiat-Shamir transform [30], the prover is able to simulate the protocol in their head and to compute an opening of the all-but-one vector commitment, such that the protocol can be used non-interactively, as a signature scheme.

3.3 FAEST algorithm

Based on the VOLEitH paradigm described in the previous section, Baum et al. [16] construct the FAEST signature scheme which uses AES as the underlying OWF.

In FAEST, the witness bits that the signer proves knowledge of using the QuickSilver proof are the secret key and the state of each round of the AES encryption. More specifically, the so-called *extended witness* contains the AES cipher key and, for each round of the key schedule, the bits that form the output of the **SubWord** operation, and for each round of the encryption, the bits that form the output of the **ShiftRows** operation.

Then, for the key schedule and for the encryption, the algorithm performs two passes each: a forward pass, which computes the bits (or their corresponding VOLE tags or keys) that form the input for each S-box, and a backward pass,

Table 1. Overview of the main parameter sets for the first-round variant of FAEST. Values $\lambda = \{128, 192, 256\}$ correspond to security levels 1, 3, and 5, respectively. Each security level features both a small (s) and fast (f) variant. For details, see [17].

Scheme	λ	l	τ	τ_0	τ_1	k_0	k_1	pk (bytes)	sig (bytes)
FAEST-128s	128	1600	11	7	4	12	11	32	5006
FAEST-128f	128	1600	16	0	16	8	8	32	6336
FAEST-192s	192	3264	16	0	16	12	12	64	12744
FAEST-192f	192	3264	24	0	24	8	8	64	16792
FAEST-256s	256	4000	22	14	8	12	11	64	22100
FAEST-256f	256	4000	32	0	32	8	8	64	28400

Algorithm 1 FAEST.KeyGen() [17]

Output: Public key pk, secret key sk

```

1: while true do
2:    $\mathbf{x} \leftarrow \{0, 1\}^{\beta \cdot 128}$   $\triangleright$  OWF input
3:    $\mathbf{k} \leftarrow \{0, 1\}^\lambda$   $\triangleright$  OWF key
4:    $\mathbf{y} := F_{\mathbf{k}}(\mathbf{x})$   $\triangleright$  OWF output
5:   if no SubBytes or SubWords has input byte {00} then
6:     return (sk := k, pk := (x, y))
```

which computes the bits (or their corresponding VOLE tags or keys) that form the output for each S-box, by applying or reversing certain AES steps. For example, the backward pass for the encryption takes the bits of the extended witness (or their corresponding VOLE tags or keys) that correspond to the output of the **ShiftRows** operation and applies an inverse **ShiftRows** operation to derive the S-box output.

These inputs and outputs are then used to compute or verify multiplicative constraints, using the approach described in Eq. 3. Specifically, for S-box input a and output b , the constraint ensures that $a \cdot b = 1$, which works because the non-linear part of the AES S-box computes a field inversion, meaning that the output b is the inverse of a . This has an added benefit of simplifying the multiplicative constraints because the values w_γ and d_γ can be omitted, as the result is constant. If during the verification all multiplicative constraints are correct, the signature is valid.

In the following we provide an overview of the key generation, signing, and verification algorithms in FAEST. As the signing and verification algorithms each make use of a larger number of subprocedures, we are not able to show these algorithms in their entirety. For a full description of all involved algorithms, we instead refer to the specification [17]. An overview of the main parameters of the algorithm is shown in Tab. 1. This paper focuses on FAEST-128f, though other variants can be approached similarly.

Key generation (Alg. 1) The key generation algorithm computes a random OWF input and OWF key. It then computes the output of the OWF under this

Algorithm 2 FAEST.Sign(sk, M) [17]**Input:** Secret key sk, message M**Output:** Signature σ

```

1:  $\rho \leftarrow \{0, 1\}^\lambda$   $\triangleright$  Deterministic variant:  $\rho \leftarrow \{0\}^\lambda$ 
2:  $\mu := H_1(\text{pk} \parallel M)$ 
3:  $(r, \text{iv}) := H_3(\text{sk} \parallel \mu \parallel \rho)$ 
4:  $(h_{\text{com}}, (\text{decom}_i)_{i \in [0, \tau)}, (\mathbf{c}_i)_{i \in [1, \tau)}, \mathbf{u}, \mathbf{V}) := \text{FAEST.VOLECommit}(r, \text{iv})$ 
5:  $\text{chall}_1 := H_2^1(\mu \parallel h_{\text{com}} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv})$ 
6:  $\tilde{\mathbf{u}} := \text{VOLEHash}(\text{chall}_1, \mathbf{u})$ 
7:  $\tilde{\mathbf{V}} := \text{VOLEHash}(\text{chall}_1, \mathbf{V})$ 
8:  $h_V := H_1(\tilde{\mathbf{V}})$ 
9:  $\mathbf{w} := \text{AES.ExtendWitness}(\text{sk}, \text{pk})$ 
10:  $\mathbf{d} := \mathbf{w} \oplus \mathbf{u}_{[0..l)}$ 
11:  $\text{chall}_2 := H_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel h_V \parallel \mathbf{d})$ 
12:  $\mathbf{u} := \mathbf{u}_{[0..l+\lambda)}, \mathbf{V} := \mathbf{V}_{[0..l+\lambda)}$   $\triangleright$  Impl. may transpose internal representation of  $\mathbf{V}$ 
13:  $(\tilde{a}, \tilde{b}) := \text{FAEST.AES.AESProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, \text{pk}, \text{chall}_2)$ 
14:  $\text{chall}_3 := H_2^3(\text{chall}_2 \parallel \tilde{a} \parallel \tilde{b})$ 
15: for  $i \in [0.. \tau)$  do
16:    $\mathbf{s}_i := \text{ChalDec}(\text{chall}_3, i)$ 
17:    $(\text{pdecom}_i) := \text{VC.Open}(\text{decom}_i, \mathbf{s}_i)$ 
18: return  $\sigma = ((\mathbf{c}_i)_{i \in [1.. \tau)}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}, (\text{pdecom}_i)_{i \in [0.. \tau)}, \text{chall}_3, \text{iv})$ 

```

input and key. As it is not possible to prove a multiplication constraint for an input that is zero using the degree-2 constraints used in FAEST¹, the algorithm ensures that no S-box in the key schedule or encryption has an input byte $\{00\}$, otherwise a new input and key is chosen. Once a suitable key and input are found, the secret key consists of the OWF key (i.e. the cipher key), and the public key consists of the OWF input and output.

Signing (Alg. 2) The signing algorithm first binds the public key to the message by deriving the value to sign, μ . This string is used alongside the secret key and additional randomness to derive the VOLEs. In addition to the VOLE random bits $\mathbf{u}$ and VOLE tag matrix $\mathbf{V}$, the VOLE commitment also computes a hash h_{com} , full decommitments decom_i for all τ parallel VOLEs and correction values $\mathbf{c}_i$ to fix all VOLE correlations to the same random bits. The first challenge chall_1 and the responses $\tilde{\mathbf{u}}$ and h_V are used to ensure the consistency of the VOLEs. The algorithm then computes the extended witness $\mathbf{w}$ by running the AES encryption and storing certain bits, and, from it, the masked witness $\mathbf{d}$, before deriving a second challenge chall_2 used to commit to the witness. It then computes the QuickSilver proof of the encryption and uses it to derive the final challenge chall_3 , which reveals the global key. Finally, using the global key, the vector commitments are opened to partial decommitments that allow recon-

¹ Baum et al. [15] later showed that multiplications with zero inputs can be proven using higher-degree constraints. This optimisation is included in the second-round submission of FAEST [14].

Algorithm 3 FAEST.Verify(M, pk, σ) [17]**Input:** Message M , public key pk , signature σ **Output:** Boolean

```

1:  $((\mathbf{c}_i)_{i \in [1..\tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}, (\text{pdecom}_i)_{i \in [0..\tau]}, \text{chall}_3, \text{iv}) = \sigma$ 
2:  $\mu := H_1(\text{pk} \parallel M)$ 
3:  $(h_{\text{com}}, \mathbf{Q}'_0, \dots, \mathbf{Q}'_{\tau-1}) := \text{FAEST.VOLEReconstruct}(\text{chall}_3, (\text{pdecom}_i)_{i \in [0..\tau]}, \text{iv})$ 
4:  $\text{chall}_1 := H_2^1(\mu \parallel h_{\text{com}} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv})$ 
5:  $\mathbf{Q}_0 := \mathbf{Q}'_0$ 
6: for  $i \in [0..\tau)$  do
7:    $b := 0$  if  $i < \tau_0$  else 1
8:    $(\delta_0, \dots, \delta_{k_b-1}) := \text{ChalDec}(\text{chall}_3, i)$ 
9:    $\tilde{\mathbf{D}}_i := [\delta_0 \cdot \tilde{\mathbf{u}} \dots \delta_{k_b-1} \cdot \tilde{\mathbf{u}}]$ 
10:  if  $i > 0$  then
11:     $\mathbf{Q}_i := \mathbf{Q}'_i \oplus [\delta_0 \cdot \mathbf{c}_i \dots \delta_{k_b-1} \cdot \mathbf{c}_i]$ 
12:  $\mathbf{Q} := [\mathbf{Q}_0 \dots \mathbf{Q}_{\tau-1}]$ 
13:  $\tilde{\mathbf{Q}} := \text{VOLEHash}(\text{chall}_1, \mathbf{Q})$ 
14:  $h_V := H_1(\tilde{\mathbf{Q}} \oplus [\tilde{\mathbf{D}}_0 \dots \tilde{\mathbf{D}}_{\tau-1}])$ 
15:  $\text{chall}_2 := H_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel h_V \parallel \mathbf{d})$ 
16:  $\tilde{b} := \text{FAEST.AES.AESVerify}(\mathbf{d}, \mathbf{Q}|_{[0..l+\lambda)}, \text{chall}_2, \text{chall}_3, \tilde{a}, \text{pk})$ 
17:  $\text{chall}'_3 := H_2^3(\text{chall}_2 \parallel \tilde{a} \parallel \tilde{b}, \lambda)$ 
18: return true if  $\text{chall}_3 = \text{chall}'_3$  else false

```

structing all-but-one vectors. The signature consists of the correction values $\mathbf{c}_i$, the VOLE hash $\tilde{\mathbf{u}}$, the masked witness bits $\mathbf{d}$, part of the QuickSilver proof $\tilde{a}$, the partial decommitments pdecom_i , the third challenge chall_3 , and an initialisation vector iv .

Verification (Alg. 3) The verification algorithm extracts the individual components of the signature. It then recomputes the signed value μ from the public key and the message, before reconstructing the VOLEs using the third challenge, the partial decommitments, and the initialisation vector. This allows the first challenge chall_1 to be recomputed. Next, the VOLE key matrix $\mathbf{Q}$ is derived by applying the correction values $\mathbf{c}_i$. To recompute the response h_V , the VOLE key column hashes must be adjusted to the witness bits by adding the VOLE hash $\tilde{\mathbf{u}}$ to the active columns. Then, the second challenge chall_2 and, from it, the second part of the QuickSilver proof, $\tilde{b}$, can be recomputed. Finally, the third challenge chall'_3 can be recomputed. If the recomputed value matches the value in the signature, the signature is valid.

4 Experimental setup

This section describes the equipment and the target implementation of FAEST used in the experiments.

4.1 Equipment

For the experiments, we use two CW308-STM32F4 target boards containing ARM Cortex-M4 STM32F415RGT6 processors running at a frequency of 24 MHz. The target boards are mounted on a CW313 adapter board. Power traces are captured and voltage fault injection is performed using a ChipWhisperer-Husky. For the side-channel attacks (i.e. the byte combine attack in Section 6 and the VOLE transpose attack in Section 7), one target is used to capture the training dataset and the other is used to capture the test dataset, i.e. the profiling for the attacks and the attacks themselves are performed on different devices, as in a real attack scenario.

For triggering of trace capture or fault injection, ARM CoreSight DWT watchpoints are used, which avoids changing the assembly instructions or register allocation of the firmware. In a real attack, other trigger sources, such as communication with peripheral devices or the sum of absolute differences between the power consumption and a reference waveform could be used instead.

4.2 Target implementation

In our experiments, we use the masked FAEST implementation by Degn et al. [25], specifically commit `b2503db`, which was the most recent at the time of the experiments. In response to our findings, the most recent commit at the time of writing (`b9cc7d5`) introduces several changes in an ongoing effort to implement shuffling and blinding as countermeasures to the byte combine and VOLE transpose attacks in Sections 6 and 7.

The implementation is compiled using `arm-none-eabi-gcc` with the highest optimization level `-O3`.

5 Physical attacks on VOLEitH signature schemes

This section describes the high-level considerations that apply to physical attacks (both side-channel analysis and fault injection) on VOLEitH-based schemes from an algorithmic perspective. Specifically, it explains how knowledge or control of certain internal variables can be utilised to recover the secret key of the signature algorithm.

Recall that the idea behind VOLEitH signature schemes is to prove the evaluation of an OWF for a known output. In the case of FAEST, the secret key is therefore simply an AES cipher key and the signature generation process is a ZKPoK of the encryption of a public plaintext under this key. The goal of an attacker is to recover the input to the OWF by means of a physical attack.

Fundamentally, there are two components of the signature generation that can be attacked. The first is the computation of the extended witness. In FAEST, this is the encryption of the public value under the cipher key where specific bits of the AES state are recorded as the witness for each round. In practice, this is implemented as a straightforward AES encryption and, as such, known attacks

and countermeasures can be applied in this context, such as [35]. Furthermore, this component appears to be under active development in the implementation by Degn et al. [25], and future changes may affect in which way physical attacks can be applied to it, if at all. We therefore do not consider such attack points in this paper and leave analysis of this component of FAEST for future work.

In other VOLEitH schemes, such as MandaRain and KuMQuat [15], the computation of the extended witness is conceptually similar and involves recording the state of the Rain block cipher [26] (MandaRain) or a solution to a system of multivariate quadratic equations (KuMQuat). In all cases, information leakage about the extended witness may be sufficient to perform secret key recovery. However, as the extended witness does not change across multiple invocations of the signing procedure, it is also possible to interpret the computation of the extended witness as a key expansion or decoding step, as found in other algorithms, such as ML-DSA or MAYO [48,19]. Future algorithms may therefore choose to only perform this step once, which limits the window of opportunity for an attacker.

The second component is the ZKPoK of the evaluation of the OWF, particularly the computations involving the VOLE correlations that encode information about the witness. The idea is to recover a subset $\{w_i \mid i \in S\}$ of the witness bits and use them to recover the OWF input. The specific witness bits that must be recovered depend on the approach used to extract the OWF input from the witness. For FAEST, this means using knowledge of parts of the AES state to recover the cipher key. For example, the first 128 bits of the witness encode the cipher key directly, thus recovering all of these bits reveals the cipher key without further processing. The next 320 bits of the witness encode the S-box output of the key schedule. These bits can be combined to recover a full round key and thus reveal the cipher key with some additional processing. The remaining 1152 bits encode the S-box output of the individual AES encryption rounds and can also be used to recover the cipher key with additional processing. In all cases, the number of witness bits required to recover the cipher key for FAEST cannot be smaller than 128 bits, regardless of the chosen method. For other VOLEitH schemes, the process is similar.

To illustrate the secret key recovery from the VOLE correlations, let $S \subseteq [0..l)$ be a subset of indices for which the corresponding witness bits $\{w_i \mid i \in S\}$ should be recovered and assume that a method for deriving the secret key from the witness bits is known. The structure given by Eq. 2 now constrains the possible attacks that result in key recovery. Clearly, revealing witness bits directly (i.e. via side-channel analysis) is sufficient to perform key recovery, while fixing their values to known values (via a fault) cannot reveal new information. However, it is also possible to recover the witness bits indirectly, i.e. through components u_i or v_i , and then derive the key. For example, recovering components u_i or fixing their values to known values allows computing $w_i := d_i + u_i$ from the public values d_i . Similarly, it is also possible to recover or fix components v_i to then compute $q'_i - v_i = \Delta \cdot w_i$ and recover w_i from the active columns. This means that

Table 2. Overview of presented attacks and their targeted components; SC = side-channel, FI = fault injection.

Type	VOLE correlation component		
	w_i	u_i	v_i
Recover	Byte combine SC attack	Counter increment skipping FI attack	VOLE transpose SC attack, Counter increment skipping FI attack
Fix	N/A ^a	VOLE conversion abort FI attack	VOLE conversion abort FI attack

^a It is not possible to reveal new information about witness bits w_i by fixing their values to known values.

all components of the VOLE correlation are relevant in the context of physical attacks.

In this paper, we present four attacks that target different components of the VOLE correlations. Tab. 2 provides an overview. The first attack, the byte combine attack in Section 6, is a side-channel attack that directly recovers a subset of witness bits. The second attack, the VOLE transpose attack in Section 7, is a side-channel attack that recovers a subset of VOLE tags that can be used to recover the witness bits. The third attack, the counter increment skipping attack in Section 8, is a fault injection attack that recovers a subset of random bits or a subset of VOLE tags by fixing a variable used to derive them. The fourth attack, the VOLE conversion abort attack in Section 9, is a fault injection attack that fixes a subset of random bits or a subset of VOLE tags to known values directly.

6 Byte combine attack

The first attack, which we call the *byte combine* attack, exploits information leakage in the `bf128.byte_combine.bits` procedure. This procedure is called from the `aes_enc_forward_128_1_round` procedure, which is part of the VOLE protocol proof for the encryption. Recall that the forward pass derives the input values to the S-boxes for each of the AES rounds and their VOLE tags. In the initial round, these input values are the XOR of the plaintext and the initial round key, which is the cipher key. The `bf128.byte_combine.bits` procedure is used to lift individual bits of the key and plaintext into the field $\mathbb{F}_{128}$, before performing the addition (i.e. XOR) to derive the S-box input values.

In the implementation of Degn et al. [25], the `bf128.byte_combine.bits` procedure uses the `bf128.mul_bit` procedure, which is the main source of leakage. In the masked version, both of these procedures are implemented in hand-written assembly, though they appear to have been derived from the C code of the reference implementation with minor modifications. For the sake of readability, this

```

1  bf128_t bf128_mul_bit(bf128_t lhs, uint8_t rhs) {
2      return bf128_and_64(lhs, -((uint64_t)rhs & 1));
3  }
4
5  bf128_t bf128_byte_combine_bits(uint8_t x) {
6      bf128_t out = bf128_from_bit(x & 1);
7      for (unsigned int i = 1; i < 8; ++i) {
8          out = bf128_add(out, bf128_mul_bit(alpha[i - 1], (x >> i) & 1));
9      }
10     return out;
11 }

```

Listing 1: The C code of the `bf128_byte_combine_bits` and `bf128_mul_bit` procedures. Sections corresponding to the assembly code in Listing 2 are highlighted in color.

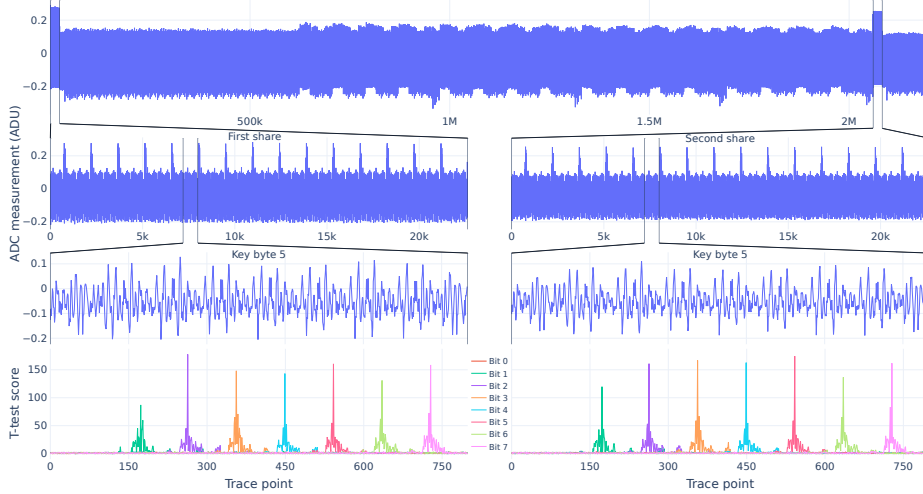


Fig. 1. Trace segmentation process and t-test results for the `bf128_byte_combine_bits` procedure, computed for 1000 traces. Note that bit 0 of each share does not produce exploitable leakage and is therefore not visible in the t-test results.

```

1  bf128_mul_bit:
2  ...
3  ldrb    r1, [sp, #20]
4  ldr     r4, [sp, #8]
5  sbfx    ip, r1, #0, #1
6  and     r2, r2, ip
7  str     r2, [r0, #0]
8  ldr     r2, [sp, #12]
9  and     r2, ip, r2
10 ...
11
12 bf128_byte_combine_bits:
13 ...
14 str     r0, [sp, #20]
15 and     r9, r1, #1
16 mov     r8, r6
17 ...
18 loop:
19 subs    r1, r4, #1
20 mov     r0, r5
21 bl      get_alpha
22 asr     r3, fp, r4
23 and     r3, r3, #1
24 str     r3, [sp, #8]
25 ...
26 bl      bf128_mul_bit
27 ...
28 bne     loop
29 ...

```

Listing 2: Simplified excerpts of the assembly code of the `bf128_mul_bit` and `bf128_byte_combine_bits` procedures. Sections corresponding to the C code in Listing 1 are highlighted in color.

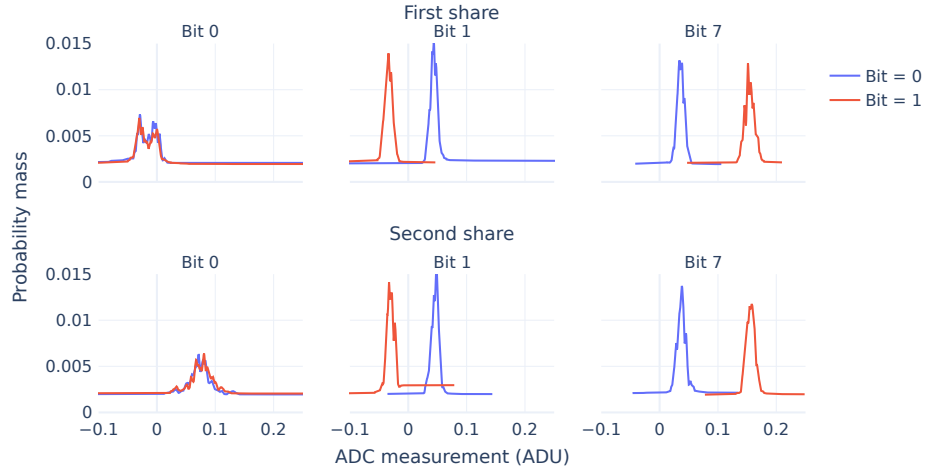


Fig. 2. Distributions of power consumption (average over 1000 traces) for bits 0, 1, and 7 at the point of maximum t-test score during the processing by `bf128_byte_combine_bits`. Note that bit 0 does not produce exploitable leakage, while the distributions for bits 1 and 7 are clearly and similarly separated between the bit values.

C code is shown in Listing 1 instead, alongside excerpts of the actual assembly code in Listing 2.

For each invocation, the input to the `bf128_byte_combine_bits` procedure is a share byte of the first round key (i.e. the cipher key). The procedure first extracts bit 0 of this value in line 6 of Listing 1, which does not produce exploitable leakage. Line 15 of Listing 2 shows that the extracted bit is kept in a register, but not written to memory (including in the parts of the assembly code that are omitted in Listing 2), which explains the lack of leakage. In the context of the attack, the least significant bit of each byte of the key thus cannot be recovered using this method and these bits are instead enumerated (at a complexity of 2^{16}).

The remaining 7 bits of each byte are then extracted one-by-one from the byte in line 8 of Listing 1, and multiplied with constant values α_i , using the `bf128_mul_bit` procedure. In the C code, this multiplication is realised by first computing a 64-bit mask, whose bits are set if the extracted bit is set or cleared if the extracted bit is not set, and then performing a bitwise AND of the mask and the value to multiply. As the Cortex-M4 is a 32-bit processor, the assembly code in lines 6 and 9 of Listing 2 instead performs two bitwise AND operations on a 32-bit mask computed in line 5 using a `sbfex` (signed bit field extract) instruction. The issue here is obvious: The Hamming weight of the mask value is 0 when the bit value is 0 and 32 when the bit value is 1. This large difference

Table 3. Neural network architectures used for the byte combine and VOLE transpose attacks.

Layer type	Output shape	
	Byte combine attack	VOLE transpose attack
Batch Normalization 1	340	512
Dense 1	128	128
Batch Normalization 2	128	128
ReLU	128	128
Dense 2	64	64
Batch Normalization 3	64	64
ReLU	64	64
Dense 3	32	32
Batch Normalization 4	32	32
ReLU	32	32
Dense 4	2	2

in Hamming weight is clearly visible in the power consumption of the device, as shown in Fig. 2 and in the t-test results in Fig. 1. By observing the power consumption, it is therefore possible to extract the witness bits (and thus the cipher key) directly.

This type of leakage was first pointed out by Amiet et al. in their attack on the key encapsulation mechanism NewHope [5], and it is also found in other PQC algorithms [55,57,41]. In all cases, the behaviour being realised is a bit-multiplication with a secret bit, which is then leaked.

A similar attack can be performed on the `aes_key_schedule_128_masked` procedure, which applies the `bf128_byte_combine_bits` procedure to derive the S-box input from the previous round key, thereby revealing the round key. As this is essentially the same attack with the same underlying leakage, we did not pursue this direction further. We believe that such an attack would succeed with a similar probability and that it can be mitigated through countermeasures in the underlying `bf128_byte_combine_bits` procedure.

6.1 Trace preprocessing and neural network training

For the profiling phase, the dataset contains 560,000 trace segments, obtained by capturing $t = 5000$ traces $\{T_1, \dots, T_t\}$ for known secret keys $\mathbf{sk}_1, \dots, \mathbf{sk}_t$ selected at random and applying the cut-and-join technique of [52]. For each trace T_i , two segments T_i^Ψ , $\Psi \in \{0, 1\}$, are extracted, containing the processing of the first and second share. Each segment is then divided into 16 subsegments $T_i^{\Psi, \eta}$, $\eta \in [0..16)$, covering the processing of one byte of the cipher key. This process is illustrated in Fig. 1 for key byte 5. Finally, each subsegment is divided into 7 further segments $T_i^{\Psi, \eta, \kappa}$, $\kappa \in [1..8)$, covering the processing of one bit of the

Algorithm 4 RecoverSecretKey($\sigma, (S_i)_{i \in R}$)

```

1: for  $k \in \{0, 1\}^{16}$  do
2:   for  $i \in [0..\lambda)$  do
3:     if  $i \bmod 8 \neq 0$  then
4:        $\text{sk}[\llbracket i \rrbracket] \leftarrow \text{argmax } S_i$ 
5:     else
6:        $\text{sk}[\llbracket i \rrbracket] \leftarrow k[\llbracket i/8 \rrbracket]$   $\triangleright$  Enumerate bit
7:   yield sk

```

cipher key (recall that the first bit, bit 0, does not produce leakage), before concatenating the segments across both shares

$$\hat{T}_i^{\eta, \kappa} := T_i^{0, \eta, \kappa} \parallel T_i^{1, \eta, \kappa}.$$

To simplify the notation, in the following, we use $\hat{T}_i^\phi$ to refer to segment $\hat{T}_i^{\eta, \kappa}$ with $\phi = 8\eta + \kappa$, i.e. we flatten the byte and bit dimensions into a single dimension. We also use the set $R := \{j \in [0..128) \mid 8 \nmid j\}$ of bits that are recoverable using this method. Each segment $\hat{T}_i^\phi$ is labelled with the value of the bit of the cipher key being processed, $\text{sk}_i[\llbracket \phi \rrbracket]$, which is the XOR of the bits being processed in each share.

The model is a multilayer perceptron (MLP) neural network with the architecture shown in Tab. 3. The network is of type $\mathcal{N} : \mathbb{R}^{340} \rightarrow \{0, 1\}$, where 340 is the number of samples in each segment. Note that Fig. 1 shows traces captured with one sample per clock cycle, but in this attack, traces were captured with four samples per clock cycle, hence the segments are longer than they appear. The network maps each segment $\hat{T}_i^\phi$ into a score vector $S_{i, \phi} = \mathcal{N}(\hat{T}_i^\phi)$, such that $S_{i, \phi}[\alpha]$ represents the probability that bit $\text{sk}_i[\llbracket \phi \rrbracket]$ of the secret key takes value $\alpha \in \{0, 1\}$:

$$S_{i, \phi}[\alpha] = \Pr[\text{sk}_i[\llbracket \phi \rrbracket] = \alpha].$$

The model is trained for a maximum of 100 epochs with early stopping at a patience of 15 using the Nadam optimiser with a learning rate of 0.01 and a numerical stability constant $\epsilon = 10^{-8}$. The batch size is 1024. Of the data 70% is used for training and 30% for validation.

6.2 Partial enumeration and secret key recovery method

As the leakage exploited in this attack directly reveals most bits of the secret key, the secret key recovery is straightforward and only requires enumeration of the missing bits, which can be done with a complexity of 2^{16} . Alg. 4 shows the main steps.

7 VOLE transpose attack

The second attack, which we call the *VOLE transpose* attack, exploits leakage during the transposition of the VOLE tag matrix in the `get_vole_aes_128_share`

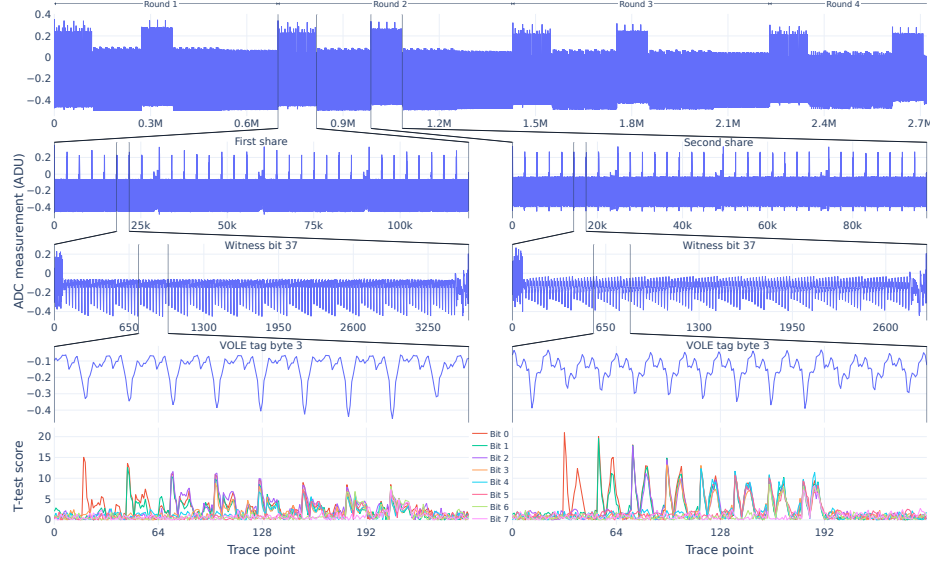


Fig. 3. Trace segmentation process and t-test results for the `get_vole_aes_128_share` procedure, computed for 1000 traces. Note the decreasing amount of leakage from bit 0 to bit 7 for each share and the overall lower amount of leakage for the first share, as indicated by the t-test results.

```

1  #define ptr_get_bit(v, i) (v[i / 8] >> (i % 8)) & 1
2  #define ptr_set_bit(d, v, i) d[i / 8] |= v << (i % 8)
3
4  bf128_t* get_vole_aes_128_share(vbb_t* vbb, int idx, int share) {
5      memset(vbb->v_buf, 0, 16);
6      uint8_t* src = share ? vbb->v_mask_cache : vbb->vole_cache;
7      uint32_t strd = share ? 1872 : 1872 / 8;
8      // Transpose on the fly into v_buf
9      for (unsigned int col = 0; col != 128; ++col) {
10         ptr_set_bit(vbb->v_buf, ptr_get_bit(src + col * strd, idx), col);
11     }
12     return (bf128_t*)vbb->v_buf;
13 }

```

Listing 3: Simplified C code of the `get_vole_aes_128_share` procedure. The section corresponding to the assembly code in Listing 4 is highlighted in colour.

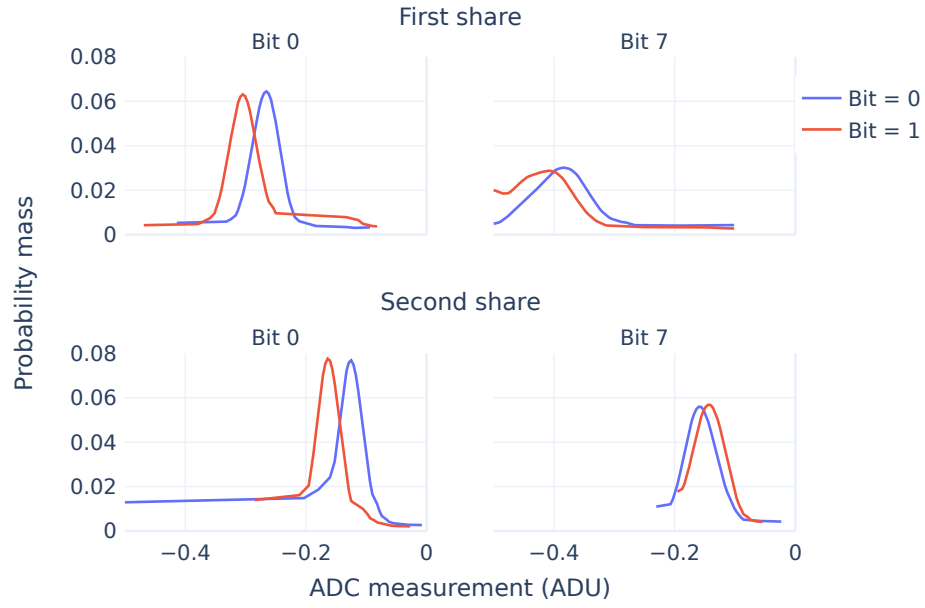


Fig. 4. Distributions of power consumption (average over 1000 traces) for bits 0 and 7 at the point of maximum t-test score during the processing by `get_vole_aes_128_share`. Note the greater separation of the distributions for bit 0 in both shares. The distributions for bit 7 of the first share are partially affected by clipping of the ADC occurring at -0.5 ADU, as a limitation of the capturing equipment.

```

1  loop:
2    ldr    r2, [r4, #16]
3    ldr    r3, [r4, #40]
4    mla    r2, r2, r1, r5
5    lsrs   r7, r2, #3
6    lsrs   r0, r1, #3
7    ldrb   r3, [r3, r7]
8    ldr    r7, [r4, #52]
9    and.w  r2, r2, #7
10   asrs   r3, r2
11   and.w  ip, r1, #7
12   ldrb   r2, [r7, r0]
13   and.w  r3, r3, #1
14   lsl.w  r3, r3, ip
15   adds   r1, #1
16   orrs   r3, r2
17   cmp    r1, r6
18   strb   r3, [r7, r0]
19   bne.n  loop

```

a) First share

```

1  loop:
2    ldr    r3, [r4, #64]
3    ldr    r5, [r4, #52]
4    ldrb   r3, [r3, r1]
5    lsrs   r0, r2, #3
6    asr.w  r3, r3, lr
7    ldrb.w ip, [r5, r0]
8    and.w  r8, r2, #7
9    and.w  r3, r3, #1
10   lsl.w  r3, r3, r8
11   adds   r2, #1
12   orr.w  r3, r3, ip
13   cmp    r2, r6
14   strb   r3, [r5, r0]
15   add    r1, r7
16   bne.n  loop

```

b) Second share

Listing 4: Simplified excerpt of the assembly code of the `get_vole_aes_128_share` procedure. Sections corresponding to the C code in Listing 3 are highlighted in colour.

procedure, called from the `aes_key_schedule_backward_128_vbb_vk_round_share` procedure, which is also part of the VOLE protocol proof for the encryption. Recall that the backward pass derives the output values of the S-boxes (and their corresponding VOLE tags) of each round of the AES key schedule.

In the implementation of Degn et al. [25], the `get_vole_aes_128_share` procedure transposes individual entries of the VOLE tag matrix $\mathbf{V}$ dynamically, using the `ptr_get_bit` and `ptr_set_bit` macros. The code for this procedure is shown in Listings 3 and 4.

For each invocation of the `get_vole_aes_128_share` procedure, the 128 bits that form the share of the VOLE tag are read from the VOLE tag matrix in non-linear order and written to the output in transposed (i.e. linear) order. For each share bit of the VOLE tag, the procedure thus reads a byte from the matrix, extracts the corresponding bit using a bitwise AND operation, and updates a byte of the output using a bitwise OR operation. These steps are also visible in the highlighted lines of Listing 4. The information leakage is primarily caused by the incremental update of the output bytes. Each `strb` operation either increases the Hamming weight of the byte by one if the corresponding bit is set or does not change the Hamming weight of the byte if the corresponding bit is not set. This slight difference in Hamming weight is visible in the power consumption of the device, as shown in Fig. 4 and in the t-test results in Fig. 3. Notably, the difference in power consumption depending on the bit value is largest for the first bit that is written and decreases for all subsequent bits as the Hamming weight (and thus the power consumption) depends on all of the previously written bits. By observing the power consumption, it is possible to extract the shares of the VOLE tags belonging to the S-box outputs, and, from a combination of both shares, compute the corresponding witness bits and, finally, the cipher key.

In practice, we observe that the leakage of the first share is slightly weaker than that of the second share. The assembly code in Listing 4 shows that, while the realised behaviour is identical between both shares, the individual instructions are similar, but not identical. Given that it is still possible to recover the bits from the first share, we did not further investigate this difference.

We further observe that the duration of the loop for the processing of the first share decreases by one clock cycle from the 74th bit onwards. This is caused by the memory layout of the STM32F415RGT6, which maps the available 128KB SRAM into a 112KB SRAM1 and a 16KB SRAM2 chip [56, p. 71, Table 3]. The `vbb->vole_cache` buffer, from which the VOLE tags for the first share are read, is allocated across this boundary, causing reads from the 74th bit onwards to be performed from SRAM2 instead. Our measurements indicate that reads from SRAM2 take one clock cycle longer than reads from SRAM1, but we hypothesise that the loop duration decreases, because, when reading from SRAM2 (and only then), the `ldrb` instruction in line 7 of the first share of Listing 4 can be pipelined together with the succeeding `ldr` instruction in line 8, which always reads from SRAM2.

In the context of the attack, it is therefore necessary to consider bits 0–72 and bits 73–127 separately. We further found that bits 0–7 of the first byte, bits 120–

127 of the last byte, as well as bits 72–79 of the tenth byte (i.e. the byte where the loop duration changes) exhibit slightly different leakage characteristics. As the number of VOLE tag bits is quite large and recovering even just a single of these bits in an active column is sufficient to recover the corresponding witness bit, we elected to discard the bytes with different leakage characteristic and instead focus on two groups of bytes: bytes 1–8 with a longer loop duration for the first share and bytes 10–14 with a shorter loop duration. Note that the second share is not affected by the changes in the loop duration.

It is worth noting that the `get_vole_aes_128_share` procedure is also used by the `bf128_sum_poly_vbb_share` procedure, which is called during the computation of v^* in the last steps of the AES proof, thereby leaking information about the last 128 VOLE tags that are used to mask the vectors of multiplicative constraints $\mathbf{a}_0$ and $\mathbf{a}_1$. The description of the multiplicative constraints in the specification [17, Section 2.3.2] states that “[...] revealing a_0 and a_1 to the verifier leaks information about the circuit values used to compute them.” However, in FAEST the multiplication checks are performed using a linear hash function, instead of using $\mathbf{a}_0$ and $\mathbf{a}_1$ directly. The signature therefore contains neither $\mathbf{a}_0$ nor $\mathbf{a}_1$, nor VOLE correlations directly involving either vector. At present, it is thus unclear how knowledge of v^* (or the VOLE tags used to compute it) can be used to recover $\mathbf{a}_0$ or $\mathbf{a}_1$. It is further also unclear whether knowledge of $\mathbf{a}_0$ or $\mathbf{a}_1$ is sufficient to reveal the secret key or to reduce the size of the search space to allow for enumeration. As the underlying leakage for this approach is the same as in the VOLE transpose attack, we expect countermeasures to be applicable to both and thus did not pursue this approach further.

7.1 Trace preprocessing and neural network training

The trace preprocessing for this attack is similar to that in the byte combine attack.

For the profiling phase, the dataset contains two groups with 5,120,000 respective 3,200,000 trace segments, obtained by capturing $t = 5000$ traces $\{T_1, \dots, T_t\}$ for known VOLE tag matrices $\mathbf{V}_1, \dots, \mathbf{V}_t$ selected at random. For each trace T_i , four segments T_i^η , $\eta \in [0..4)$ are extracted, each containing a single round of the key schedule. Note that the AES-128 key schedule actually performs a total of 10 rounds. For the attack, it is only important that four consecutive rounds are recovered (to give a total of $4 \cdot 32 = 128$ bits of information), so an attacker can theoretically make multiple attempts at recovering the secret key by recovering different sets of consecutive rounds from a single trace. In practice, we found it sufficient to recover only the first four rounds. The segments are then split into segments $T_i^{\eta,\Psi}$, $\Psi \in \{0,1\}$, containing the processing of the first and second share within each round. Each segment is then divided into 32 further segments $T_i^{\eta,\Psi,\kappa}$, $\kappa \in [0..32)$, covering the processing of one witness bit. Finally, each segment is divided into 16 segments $T_i^{\eta,\Psi,\kappa,\nu}$, $\nu \in [0..16)$, covering the processing of one byte of the VOLE tag.

This process is illustrated in Fig. 3. As the leakage is clearly dependent on the value of previous bits, individual bits are not extracted into their own segments.

Instead, segments

$$\hat{T}_i^{\eta,\kappa,\nu} := T_i^{\eta,0,\kappa,\nu} \parallel T_i^{\eta,1,\kappa,\nu}$$

are again concatenated across shares. To simplify the notation, we use $\hat{T}_i^{\phi,\pi} := \hat{T}_i^{\eta,\kappa,\nu}$ with $\phi = 32\eta + \kappa$ and $\pi = \nu$, i.e. we flatten the round and witness dimensions into a single dimension. Recall that the rounds of the key schedule leak the S-box output for the computation of the round keys, which, for the first four rounds, is stored in bits 128–255 of the extended witness. The corresponding VOLE tags thus occupy rows 128–255 of the VOLE tag matrix, so in a further simplification of notation, we consider vectors $\hat{\mathbf{v}}_i$ of VOLE tags with $\hat{\mathbf{v}}_i[\iota] := \text{ToField}(\mathbf{V}_i|_{128+\iota})$ for $\iota \in [0..128)$ (i.e. we interpret rows 128–255 as a vector of field elements). Each segment $\hat{T}_i^{\phi,\pi}$ is labelled with the 8 bits of the VOLE tag being processed, $\hat{\mathbf{v}}_i[\phi][8\pi..8\pi+7]$, which is the XOR of the 8 bits being processed in each share.

As explained previously, we discard bytes 0, 9, and 15, as they exhibit different leakage characteristics. We thus form groups

$$\begin{aligned} G_0 &:= \{\hat{T}_i^{\phi,\pi} \mid \pi \in [1..8]\}, \\ G_1 &:= \{\hat{T}_i^{\phi,\pi} \mid \pi \in [10..14]\}. \end{aligned}$$

In the following, we will also use the set

$$R := \{8x + y \mid x \in [0..16), y \in [0..8), x \notin \{0, 9, 15\}\}$$

of VOLE tag matrix columns that are recoverable using this method.

In total, we train 16 MLP neural networks $\mathcal{N}_{g,n} : \mathbb{R}^{512} \rightarrow \{0, 1\}$, $g \in \{0, 1\}, n \in [0..8)$, with the architecture shown in Tab. 3, one for each bit of a byte in both of the groups. The networks map each segment $\hat{T}_i^{\phi,\pi} \in G_g$ into a score vector $S_{i,\phi,8\pi+n} = \mathcal{N}_{g,n}(\hat{T}_i^{\phi,\pi})$, such that $S_{i,\phi,8\pi+n}[\alpha]$ represents the probability that the VOLE tag bit $\hat{\mathbf{v}}_i[\phi][8\pi + n]$ takes value $\alpha \in \{0, 1\}$:

$$S_{i,\phi,8\pi+n}[\alpha] = \Pr[\hat{\mathbf{v}}_i[\phi][8\pi + n] = \alpha]. \quad (4)$$

The models are trained using the same strategy and parameters as in the byte combine attack.

7.2 Partial enumeration and secret key recovery method

Unlike in the byte combine attack, knowledge of the VOLE tags does not reveal the secret key directly. In this section, we describe an approach for secret key recovery that combines information about several VOLE tags and uses the maximum predicted class probability to select certain bits for enumeration, thereby increasing the success probability of the attack.

Alg. 5 shows the steps of the approach. As input, the algorithm receives the signature σ and a list of predictions $(S_{i,j})_{i \in [0..\lambda), j \in R}$ (note that Eq. 4 implicitly flattens the VOLE tag byte and bit dimensions into a single dimension indexed

Algorithm 5 RecoverSecretKey($\sigma, (S_{i,j})_{i \in [0..\lambda], j \in R}$)

```

1:  $((\mathbf{c}_i)_{i \in [1..\tau]}, (\mathbf{pdecom}_i)_{i \in [1..\tau]}, \mathbf{chall}_3, \mathbf{iv}) = \sigma$ 
2:  $(\mathbf{Q}, \Delta) \leftarrow \text{VOLEReconstructAndCorrect}(\mathbf{chall}_3, \mathbf{iv}, (\mathbf{pdecom}_i)_{i \in [1..\tau]}, (\mathbf{c}_i)_{i \in [1..\tau]}, \hat{l})$ 
3:  $R_\Delta \leftarrow R \cap \{i \in [0..\lambda] \mid \Delta[i] = 1\}$   $\triangleright$  Valid active columns
4: for  $i \in [0..\lambda]$  do
5:    $\mathbf{q}[i] \leftarrow \text{ToField}(\mathbf{Q}_{|\lambda+i})$ 
6: for  $i \in [0..\lambda]$  do
7:   for  $j \in R_\Delta$  do
8:      $\mathbf{w}[j] \leftarrow \mathbf{q}[i][j] \oplus \text{argmax}(S_{i,j})$ 
9:      $m_0 \leftarrow \sum_{j \in R_\Delta} (1 - \mathbf{w}[j]) \cdot \max(S_{i,j}) + \mathbf{w}[j] \cdot \min(S_{i,j})$ 
10:     $m_1 \leftarrow \sum_{j \in R_\Delta} \mathbf{w}[j] \cdot \max(S_{i,j}) + (1 - \mathbf{w}[j]) \cdot \min(S_{i,j})$ 
11:     $\mathbf{b}[i] \leftarrow \text{argmax}(\{m_0, m_1\})$   $\triangleright$  Witness bit prediction
12:     $\mathbf{p}[i] \leftarrow \max(\{m_0, m_1\}) / (m_0 + m_1)$   $\triangleright$  Prediction probability
13:  $e \leftarrow \text{argsort}(\mathbf{p})_{[0..n_{\text{enum}}]} \cap \{i \in [0..\lambda] \mid \mathbf{p}[i] \leq 0.9\}$ 
14: for  $k \in \{0, 1\}^{|e|}$  do
15:   for  $i \in [0..|e|]$  do
16:      $\mathbf{b}[e[i]] \leftarrow k[i]$   $\triangleright$  Enumerate bits
17:      $k'_4 \leftarrow \mathbf{b}_{[0..32]}, k'_8 \leftarrow \mathbf{b}_{[32..64]}, k'_{12} \leftarrow \mathbf{b}_{[64..96]}, k'_{16} \leftarrow \mathbf{b}_{[96..128]}$ 
18:      $k'_7 \leftarrow \text{SubWord}^{-1}(\text{RotWord}^{-1}(k'_8 \oplus k'_4 \oplus \text{Rcon}[1]))$ 
19:      $k'_{11} \leftarrow \text{SubWord}^{-1}(\text{RotWord}^{-1}(k'_{12} \oplus k'_8 \oplus \text{Rcon}[2]))$ 
20:      $k'_{15} \leftarrow \text{SubWord}^{-1}(\text{RotWord}^{-1}(k'_{16} \oplus k'_{12} \oplus \text{Rcon}[3]))$ 
21:      $k'_{10} \leftarrow k'_{11} \oplus k'_7$ 
22:      $k'_{14} \leftarrow k'_{15} \oplus k'_{11}$ 
23:      $k'_{13} \leftarrow k'_{14} \oplus k'_{10}$   $\triangleright$  Fourth round key fully recovered
24:     for  $i \in [2..0]$  do  $\triangleright$  Run key schedule in reverse to get original cipher key
25:        $k'_{4i+0} \leftarrow k'_{4(i+1)+0} \oplus \text{SubWord}(\text{RotWord}(k'_{4(i+1)+3} \oplus k'_{4(i+1)+2} \oplus \text{Rcon}[i]))$ 
26:        $k'_{4i+1} \leftarrow k'_{4(i+1)+1} \oplus k'_{4(i+1)+0}$ 
27:        $k'_{4i+2} \leftarrow k'_{4(i+1)+2} \oplus k'_{4(i+1)+1}$ 
28:        $k'_{4i+3} \leftarrow k'_{4(i+1)+3} \oplus k'_{4(i+1)+2}$ 
29:   yield  $\mathbf{sk} := k'_0 \parallel k'_1 \parallel k'_2 \parallel k'_3$ 

```

by j and that i now indexes the witness bit, not the trace). From there, the VOLE key matrix $\mathbf{Q}$ is reconstructed, using the `VOLEReconstructAndCorrect` procedure, which is equivalent to steps 3 to 12 of Alg. 3.

Then, using the models' predictions $S_{i,j}$, for each witness bit, the predicted class probabilities for witness bit value 0 and 1 are added up across all VOLE tag bits in the active columns. This requires adjusting the predicted class probabilities such that they correspond to the resulting witness bit, not the VOLE tag. For example, a VOLE tag prediction of 1 for a VOLE key of 1 corresponds to a witness bit prediction of 0, not 1, due to Eq. 2. The final witness bit prediction is then given by whether the total predicted class probability for the witness bit is larger for bit value 0 or 1. We found that this approach better accounts for VOLE tag predictions with lower maximum predicted class probability, which would otherwise be overrepresented in a majority vote on bit values alone. A normalised probability of the witness bit prediction (i.e. a probability in $(0.5, 1.0]$ indicating the overall confidence in the witness bit prediction) is also computed.

```

1 void prg(uint8_t* key, uint8_t* iv, uint8_t* out, size_t outlen) {
2     aes128ctx_publicinputs ctx128;
3     aes128_ecb_keyexp_publicinputs(&ctx128, key);
4     for (; outlen >= 16; outlen -= 16, out += 16) {
5         aes128_ecb_publicinputs(out, iv, 1, &ctx128);
6         aes_increment_iv(iv);
7     }
8     aes128_ctx_release_publicinputs(&ctx128);
9 }

```

Listing 5: The C code of the `prg` procedure. The section corresponding to the assembly code in Listing 6 is highlighted in colour.

```

1  loop:
2      mov r0, r4
3      mov r3, r5
4      movs    r2, #1
5      mov r1, sp
6      bl  aes128_ecb_publicinputs
7      adds    r4, #16
8      mov r0, sp
9      bl  aes_increment_iv
10     cmp r4, r7
11     bne.n   loop
12     ...

```

Listing 6: Simplified excerpt of the assembly code of the `prg` procedure. The section corresponding to the C code in Listing 5 is highlighted in colour.

For enumeration, up to $n_{\text{enum}} = 32$ bits with lowest normalised probability (up to a normalised probability of 0.9) are selected and the corresponding witness bits are enumerated. The value 0.9 was chosen empirically to reduce the average enumeration cost.

The resulting witness prediction then gives the words that form the S-box output for round keys 1–4 during the key schedule. From this, all 4 words of round key 4 are recomputed, which allows applying the key schedule in reverse to derive the initial round key, which is the cipher key.

8 Counter increment skipping attack

The third attack, which we call the *counter increment skipping* attack, skips the incrementation step for the counter in the pseudorandom generator `prg` used to derive the per-tree keys from the root key during the opening of the vector commitment.

Recall that, instead of computing a single large VOLEitH instance, FAEST computes $\tau = 16$ different VOLEitH instances and then combines these to construct the final VOLE correlation. The individual VOLEitH instances each use vector commitments computed using a GGM tree. To derive the keys that are used as the roots of the GGM trees, FAEST expands a so-called *root key* into several individual keys using a pseudorandom generator, which is AES in CTR mode. The pseudorandom generation of the expanded keys is therefore performed by encrypting an incrementing counter under the root key and using each block of output as a root for a GGM tree.

The idea behind the attack is to skip the incrementation of the counter with a single fault during the expansion of the keys. This causes the pseudorandom generator to generate the same output for two blocks, meaning that two neighbouring GGM trees are derived from the same root. As the nodes of the GGM tree are derived pseudorandomly from the root, the leaves of these two trees, i.e. the seeds sd_i , will be the same. For both trees, the value of the global key Δ will now choose all-but-one of these seeds to reveal publicly in the signature. If the value that is missing in the first tree is revealed in the second tree, it is possible to recompute the entire GGM tree and thereby break the hiding property of the VOLE commitment. Knowledge of the full tree is sufficient to derive the values u_i or v_i , and, from there, recover the witness bits from the signature. In cases where the same seed sd_i is missing from both trees (i.e. $\Delta[0..7] = \Delta[8..15]$), it is not possible to recover a tree and the attack fails. As the value of Δ is uniformly and randomly distributed, the case where the same seed is selected in both trees occurs with the probability of approximately 2^{-8} and is therefore not a problem in practice.

In the implementation of Degn et al. [25], the computation of the GGM trees is performed twice, once to compute all VOLE tags prior to the VOLE protocol proof using the `partial_vole_commit_column` procedure and once to compute each GGM tree individually during the opening of the vector commitment at the end of the signature generation using the `vector_open_ondemand` procedure. Fixing the roots of the GGM trees during the computation of the VOLE tags therefore does not lead to key recovery, because the partial decommitments in the signature are recomputed from the original root key during the opening of the vector commitment and are not faulty. We instead target the pseudorandom generator procedure `prg` called in the `vector_open_ondemand` procedure. By skipping the branching to the `aes_increment_iv` procedure in line 6 of Listing 5 (i.e. by skipping the `b1` instruction in line 9 of Listing 6), the `prg` procedure will generate two identical blocks of output, leading to two identical GGM trees.

There is also a variant of this attack that instead targets the computation of child nodes within a tree, thereby creating identical subtrees that can be used for key recovery. This variant of the attack is also applicable to the updated batch all-but-one vector commitment used for the second-round variant of FAEST and is described in more detail in Section 12.1. As the underlying vulnerability in the `prg` procedure is the same for both attacks, we did not pursue this approach further.

Algorithm 6 RecoverSecretKey(σ)

```

1:  $(\mathbf{d}, (\text{pdecom}_i)_{i \in [1..\tau]}, \text{chall}_3, \text{iv}) \leftarrow \sigma$ 
2:  $(\delta_{0,0}, \dots, \delta_{0,k_1-1}) \leftarrow \text{ChalDec}(\text{chall}_3, 0)$ 
3:  $(\delta_{1,0}, \dots, \delta_{1,k_1-1}) \leftarrow \text{ChalDec}(\text{chall}_3, 1)$ 
4:  $\Delta_0 \leftarrow \text{NumRec}((\delta_{0,0}, \dots, \delta_{0,k_1-1}))$ 
5:  $\Delta_1 \leftarrow \text{NumRec}((\delta_{1,0}, \dots, \delta_{1,k_1-1}))$ 
6: if  $\Delta_0 = \Delta_1$  then return  $\perp$   $\triangleright$  Cannot recover if same seed is missing
7:  $(s_{0,j})_{j \in [0..2^{k_1}], j \neq \Delta_0} \leftarrow \text{VC.Reconstruct}(\text{pdecom}_0, (\delta_{0,0}, \dots, \delta_{0,k_1-1}), \text{iv})$ 
8:  $(s_{1,j})_{j \in [0..2^{k_1}], j \neq \Delta_1} \leftarrow \text{VC.Reconstruct}(\text{pdecom}_1, (\delta_{1,0}, \dots, \delta_{1,k_1-1}), \text{iv})$ 
9: for  $j \in [1..2^{k_1}]$  do  $\text{sd}_j := s_{0,j \oplus \Delta_0}$   $\triangleright$  Use missing seed from second tree
10:  $\text{sd}_0 := s_{1,\Delta_0}$ 
11:  $\mathbf{u} \leftarrow \text{ConvertToVOLE}(\text{sd}_0, \dots, \text{sd}_{2^{k_1}-1}, \text{iv}, \hat{l})$ 
12:  $\mathbf{w} \leftarrow \mathbf{d}_{[0..\lambda)} \oplus \mathbf{u}_{[0..\lambda)}$ 
13: return  $\text{sk} := \mathbf{w}$ 

```

8.1 Secret key recovery method

Recovering the seed from a signature with two identical GGM trees is straightforward. Alg. 6 shows the steps of the approach. If the same seed is missing in both trees, the recovery fails. Otherwise, the seeds of the individual trees are reconstructed, the missing seed in the first tree is taken from the second tree, and the vector $\mathbf{u}$ is recomputed from the now complete set of seeds. Using $\mathbf{u}$ and the vector $\mathbf{d}$ from the signature, the witness $\mathbf{w}$ is recovered and the secret key (i.e. the first 128 bits) is extracted.

Note that it is also possible to compute a VOLE tag matrix from the full set of seeds and to recover $\mathbf{w}$ from the VOLE correlations. This approach works in the same cases, because $\Delta_0 \neq \Delta_1$ implies that at least one column in the VOLE tag matrices derived from the first or second tree is active. As it is also possible to take the missing seed in the second tree from the first tree (as opposed to the other way around), both VOLE tag matrices can be computed. Thus, a VOLE tag for an active column is known and can be used to recover $\mathbf{w}$. However, as there is no apparent benefit to this slightly more complicated approach, we only present the recovery from $\mathbf{u}$.

9 VOLE conversion abort attack

The fourth attack, which we call the *VOLE conversion abort* attack, targets the conversion of the first GGM tree seeds to the vector $\mathbf{u}_0$ of random bits and the VOLE tag matrix $\mathbf{V}_0$ during the VOLE commitment.

The idea behind the attack is to abort the conversion of the seeds to the VOLE so that the output is dependent only on the first seed. In cases where this seed is revealed in a partial decommitment in the signature (i.e. where $\Delta[0..7] \neq \mathbf{0}$), $\mathbf{u}_0$ and $\mathbf{V}_0$ are thereby fixed to known values that can be recomputed to reveal the secret key. In cases where the first seed is missing (i.e. where $\Delta[0..7] = \mathbf{0}$),

```

1 void partial_vole_commit_column(uint8_t* rootKey, uint8_t* iv,
2                               vole_t* out) {
3     uint8_t* expanded_keys[16*16];
4     prg(rootKey, iv, expanded_keys, 128, 16*16);
5     vec_com_t vec_com;
6     int factor_32 = 234 / 4;
7     for (unsigned int idx = 0; idx < 16; idx++) {
8         vector_commitment(expanded_keys + idx*16, &vec_com);
9         vole_t* cur_vole = out + idx*(sizeof vole_t);
10        uint8_t* u = cur_vole->u;
11        uint8_t* c = cur_vole->c;
12        uint8_t* v = cur_vole->v;
13
14        for (unsigned int i = 0; i < 256; i++) {
15            extract_sd_com(&vec_com, iv, i, sd, com);
16            prg(sd, iv, r, 128, 234);
17
18            // Compute u
19            xor_u32_array(u, r, u, factor_32);
20            xor_u8_array(u + factor_32*4, r + factor_32*4,
21                       u + factor_32*4, 234 - factor_32*4);
22
23            // Compute VOLE tags
24            for (unsigned int j = i*8; j < (i+1)*8; j++) {
25                unsigned int t_v = j - i*8;
26                if ((i >> t_v) & 1) {
27                    xor_u32_array(v + j, r, v + j, factor_32);
28                    xor_u8_array(v + j + factor_32*4, r + factor_32*4,
29                               v + j + factor_32*4, 234 - factor_32*4);
30                }
31            }
32        }
33
34        if (idx != 0) {
35            // Compute correction values
36            xor_u8_array(out->u, u, c, 234);
37        }
38    }
39 }

```

Listing 7: The C code of the `partial_vole_commit_column` procedure. The section corresponding to the assembly code in Listing 8 is highlighted in colour.

1	...
2	ldr r3, [r7, #112]
3	adds r4, #1
4	cmp r3, r4
5	bhi.w loop
6	mov fp, r6
7	ldr r3, [r7, #124]
8	...

Listing 8: Simplified excerpt of the assembly code of the `partial_vole_commit_column` procedure. The section corresponding to the C code in Listing 7 is highlighted in colour.

Algorithm 7 RecoverSecretKey(σ)

```

1:  $(\mathbf{d}, (\text{pdecom}_i)_{i \in [1..\tau]}, \text{chall}_3, \text{iv}) \leftarrow \sigma$ 
2:  $(\delta_{0,0}, \dots, \delta_{0,k_1-1}) \leftarrow \text{ChalDec}(\text{chall}_3, 0)$ 
3:  $\Delta_0 \leftarrow \text{NumRec}(k_1, (\delta_{0,0}, \dots, \delta_{0,k_1-1}))$ 
4: if  $\Delta_0 = 0$  then return  $\perp$   $\triangleright$  Cannot recover if first seed is missing
5:  $(s_j)_{j \in [0..2^{k_1}], j \neq \Delta_0} \leftarrow \text{VC.Reconstruct}(\text{pdecom}_0, (\delta_{0,0}, \dots, \delta_{0,k_1-1}), \text{iv})$ 
6:  $\mathbf{u} \leftarrow \text{PRG}(s_0, \text{iv}, l)$ 
7:  $\mathbf{w} \leftarrow \mathbf{d}_{[0..\lambda)} \oplus \mathbf{u}_{[0..\lambda)}$ 
8: return  $\text{sk} := \mathbf{w}$ 

```

the attack fails, which occurs with approximate probability 2^{-8} and is therefore not a problem in practice.

In the implementation by Degn et al. [25], the VOLE conversion and commitment are merged into the `partial_vole_commit_column` procedure. By aborting the loop in line 14 of Listing 7 (i.e. by skipping the `bhi` instruction in line 5 of Listing 8) during the VOLE commitment to the first GGM tree, the vector $\mathbf{u}_0$ of random bits and the VOLE tag matrix $\mathbf{V}_0$ depend only on the first seed and can thus be recomputed. Note that it would also be possible to abort the loop during the VOLE commitment for a different GGM tree k and use either its VOLE tag matrix $\mathbf{V}_k$ or recompute the original vector $\mathbf{u}_0$ from the correction values $\mathbf{c}_k = \mathbf{u}_0 \oplus \mathbf{u}_k$ and knowledge of $\mathbf{u}_k$.

We further found that it is occasionally possible to skip the loop entirely, thereby causing the value of $\mathbf{u}_0$ to be zeroised (the implementation explicitly zero-initialises this value). However, this fault only succeeds with very low probability and causes the device to crash if it fails.

9.1 Secret key recovery method

Recovering the secret key from a signature with a partial VOLE is similar to the recovery in the counter increment skipping attack. Alg. 7 shows the steps of the approach. If the first seed of the first GGM tree is missing, the recovery

Table 4. Empirical success probabilities (approximate 95% confidence interval estimate) and enumeration complexities for 1000 secret keys selected at random.

Byte combine attack	VOLE transpose attack	Counter increment skipping attack	VOLE conversion abort attack
0.998 ± 0.003 (2^{16})	1.0 ± 0.0 (0)	0.952 ± 0.013 (0)	0.871 ± 0.021 (0)

fails. Otherwise, the seeds of the first tree are reconstructed. The vector $\mathbf{u}$ can then be derived from the first seed using the PRG pseudorandom generator, and, like in the previous attack, can be used to recover the witness $\mathbf{w}$ and the secret key. Note that the implementation by Degn et al. [25] does not reorder the seeds during the VOLE commitment. The output of the `ConvertToVOLE` procedure is independent of the order of the individual seeds if all seeds are provided, thus the lack of reordering has no effect on the generated signature. However, in the context of the attack, it is important that all seeds used prior to the VOLE conversion being aborted are known. Thus, if the implementation had reordered the seeds, aborting the loop after the first iteration would not have resulted in secret key recovery, because the needed seed (i.e. the seed at index Δ_0) would have been missing from the signature.

As in the counter increment skipping attack, it is also possible to compute the VOLE tag matrix and recover $\mathbf{w}$ from the VOLE correlations. Again, this approach works in the same cases, because $\Delta_0 \neq 0$ implies that at least one column in the VOLE tag matrix derived from the first tree is active. Thus, recomputing the VOLE tag matrix from the first seed gives a VOLE tag in an active column that can be used to recover $\mathbf{w}$. As before, there is no apparent benefit to this slightly more complicated approach, which is why we only present the recovery from $\mathbf{u}$.

10 Experimental results

This section describes the results of the presented side-channel and fault injection attacks. The stated probabilities are empirical probabilities (mean over 1000 secret keys selected at random) for recovering the secret key from a single execution of the signing procedure. The results are summarised in Tab. 4.

For the byte combine attack, we train a neural network as described in Section 6.1. Using this network and the technique described in Section 6.2, we are able to recover the secret key in 998 of 1000 attempts, with an enumeration of 2^{16} for the bits that cannot be recovered using the attack. This gives an approximate 95% confidence interval estimate of the success probability of 0.998 ± 0.003 .

For the VOLE transpose attack, we train 16 neural networks as described in Section 7.1. Using these networks and the technique described in Section 7.2, we are able to recover the secret key in 1000 of 1000 attempts even without the use of enumeration (i.e. with $n_{\text{enum}} = 0$ in Alg. 5). This gives an approximate 95% confidence interval estimate of the success probability of 1.0 ± 0.0 . Clearly, the

limited size of our test dataset causes us to overestimate the success probability. In a larger dataset, direct secret key recovery may fail for certain traces, in which case the presented enumeration technique may be able to compensate and allow the key to still be recovered.

For the counter increment skipping attack, we are able to recover the secret key in 952 of 1000 attempts using the technique described in Section 8.1. This gives an approximate 95% confidence interval estimate of the success probability of 0.952 ± 0.013 . Among the unsuccessful attempts, we encounter the scenario where the same seed is missing from both GGM trees for three signatures.

For the VOLE conversion abort attack, we are able to recover the secret key in 871 of 1000 attempts using the technique described in Section 9.1. This gives an approximate 95% confidence interval estimate of the success probability of 0.871 ± 0.021 . Among the unsuccessful attempts, we encounter the scenario where the first seed is missing from the first GGM tree for two signatures.

11 Countermeasures

This section discusses potential countermeasures against the presented side-channel and fault injection attacks.

11.1 Countermeasures common to both side-channel attacks

One approach to protect against the byte combine attack and the VOLE transpose attack is to remove the underlying bitwise leakage by parallelising operations. In both attacks, the bit-by-bit processing allows secrets to be extracted, even in the presence of masking. As shown in [27], higher-order masking countermeasures are not effective unless the bitwise leakage is eliminated. This may be achieved by bitslicing the operations which need to be protected, to ensure that multiple bits are processed in parallel.

A different approach is to employ shuffling [21] to randomly reorder operations at runtime. While this approach still allows an attacker to identify the value of bits being processed, it makes it harder for the attacker to map these bits into the secrets. It is important to ensure that the shuffling procedure is itself resistant to physical attacks, such as [11,40], and that the number of operations being shuffled is sufficiently large to not be enumerable.

11.2 Countermeasures against the byte combine attack

In the paper [5] first identifying this type of leakage in NewHope, several countermeasures were proposed. One approach involves minimising the leakage by reducing the number of bits in the generated mask. This approach is also applicable to the `bf128_byte_combine_bits` procedure, as the left operands of the bit multiplications are constant generator values α_i with known bit-patterns. Though, as pointed out in [5], even a smaller mask can generate sufficient leakage to enable an attack, so this countermeasure alone may not be sufficient in practice.

The authors of [5] also suggest the use of shuffling, though the approach they present for NewHope of reordering the multiplications in the loop cannot be translated to FAEST directly, because the `bf128_byte_combine_bits` procedure processes individual bytes and the seven bit operations that could be reordered in this way are easily enumerable. Instead, the shuffling should be performed at a higher level, across both bytes and bits.

Note that the byte combine attack would not be prevented by using the Even-Mansour (EM) variants of FAEST that instead use AES with a public key and secret input (see [17]), as the implementation by Degn et al. [25] processes the input using the same `bf128_byte_combine_bits` function as the one used for the key.

11.3 Countermeasures against the VOLE transpose attack

As mentioned previously, shuffling the order in which bits are read and written during the transpose prevents the VOLE transpose attack. When implementing such a type of shuffling for this procedure, care must be taken to shuffle both the order of witness bits and the order of VOLE tag matrix columns that are being processed, as the Hamming weight of a VOLE tag (or parts of it) is sufficient for distinguishing a witness bit, so it is not enough to simply reorder the VOLE tag matrix columns.

In fact, we believe that protecting computations involving VOLE tags will be the central challenge for VOLEitH schemes, because even imprecise information (such as the Hamming weight of a byte or word) can be used to recover the key and preventing leakage of the Hamming weight is difficult. Both bitslicing and shuffling can effectively reduce the ability of an attacker to recover this information.

We also experimented with using the bit-banding feature available in some Cortex-M4 processors [56, Section 2.3.3]. This feature maps part of the original address space into an alias region, such that individual bits in the original address space are word-addressable in the alias region. This allows eliminating the bitwise arithmetic operations used to extract and insert individual bits during the transpose. However, we found that storing individual bits to memory still produced sufficiently strong leakage to distinguish individual bit values.

Note that this attack would also not be prevented by using the EM variants of FAEST, as the `get_vole_aes_128_share` procedure in the implementation by Degn et al. [25] is also used for the combined forward and backward pass for the proof of the encryption, which leaks VOLE tags for witness bits of the encryption.

11.4 Countermeasures common to both fault attacks

To protect against both the counter increment skipping attack and the VOLE conversion abort attack, it is possible to check the validity of the generated signature. In both cases, the attacks cause the generated signature to be invalid, which can be detected and used to abort the signing procedure without revealing the

signature, thereby preventing the attacks. Note that, for the counter increment skipping attack, this is only possible due to how the implementation by Degn et al. [25] handles the opening of the vector commitments. As described in Section 8, in the implementation, the opening procedure recomputes the expanded keys and GGM trees from a root key. A fault during the opening therefore causes the partial decommitments to not match the VOLE correlations used to compute the rest of the signature. In an implementation that only computes the GGM trees once, the VOLE correlations would match the partial decommitments and the attack could thus not be detected in this way. The VOLE conversion abort attack always causes the generated signature to become invalid, even without repeating computations.

More broadly, both attacks could also be prevented by eliminating the branches that are targeted by the fault injection. For the counter increment skipping attack, this would involve inlining the `aes_increment_iv` procedure into the `prg` procedure (or combining it with the encryption). As the `aes_increment_iv` procedure is relatively simple, this approach would be an effective way of preventing the attack. For the VOLE conversion abort attack, this would require unrolling the loop that is targeted by the fault injection. Due to the large number of iterations of this loop (256 for FAEST-128f), this approach is likely to be practically infeasible.

11.5 Countermeasures against the counter increment skipping attack

One approach to prevent the counter increment skipping attack is to check for identical GGM trees during the signature generation. This can be accomplished, for example, by explicitly checking the expanded keys r_i used to perform the vector commitments, or by comparing the computed commitments `comi` during the VOLE commitment. It is also possible to use the vectors of correction values c_i to detect identical trees. If the first and second tree are identical, the first set of correction values is all-zero. If two other neighbouring trees are identical, their correction values are identical. If identical trees are detected, the signing should be aborted without revealing the signature, thereby preventing the attack.

A different approach is to eliminate the use of multiple GGM trees entirely by replacing them with a batch all-but-one vector commitment, as proposed in [15]. Their approach instead uses a single larger tree to derive the seeds used to compute the VOLE correlations, thereby eliminating the need to expand a root key into several keys and thus preventing the attack. They also introduce a per-node tweak to the derivation, which causes subtrees computed from the same value to be different, as they are in different positions in the tree. However, in Section 12.1, we describe how to adapt the attack to batch commitments by targeting the derivation of child nodes within the tree. A better approach may thus be to apply the half-tree technique [34,24,20], which is discussed in Section 12.3.

Yet another approach is to replace the underlying PRG (which is AES in CTR mode) with a different PRG, such as SHAKE [46]. The sponge construc-

tion used in SHAKE does not require the use of an incrementing counter, thereby preventing the attack. However, it has been shown [39,38,58] that it is possible to control the output of SHAKE via a single fault injection, thus it is unclear whether such an approach would be effective. This is discussed further in Section 12.4 in the context of signature schemes PERK, LESS and CROSS, which employ such a construction.

11.6 Countermeasures against the VOLE conversion abort attack

As mentioned in Section 9.1, reordering the seeds during the VOLE conversion in the VOLE commitment to the ordering dictated by the specification [17] effectively prevents the attack. The reason for this is that the loop abort causes the faulty VOLE to depend only on the first seed. If this seed is missing from the signature (i.e. if the missing seed is processed first), it is not possible to recompute the faulty VOLE and recover the secret key. Note that shuffling the order in which the VOLEs are processed during the conversion is not enough to prevent the attack, as all-but-one of the possible seeds are included in the signature and the value of the faulty VOLE could thus easily be enumerated.

It is also possible to use a similar approach to the detection of identical GGM trees for the counter increment skipping attack involving the correction values. In the implementation by Degn et al. [25], the buffer containing the correction values is explicitly zero-initialised. Thus, when aborting the loop, the correction values computed in later iterations will instead be zero, which can be detected directly or through the comparison of neighbouring correction values proposed in the previous section. If a loop abort is detected in this way, the signing should be aborted without revealing the signature, thereby preventing the attack.

12 Applicability of the presented attacks to the second-round submissions

As mentioned previously, the second-round submissions to the competition introduced a number of changes for four of the five MPCitH-based signature schemes that more closely align these algorithms with the VOLEitH paradigm (and thus FAEST). In this section we discuss whether and how the presented attacks can be extended to the updated submissions for FAEST [14], Mirath [1], MQOM [18], RYDE [6], and SDitH [2]. We also briefly comment on the applicability of one of the extended attacks on the remaining MPCitH-based signature scheme PERK [7], and code-based schemes CROSS [12] and LESS [13]. To the best of our knowledge, there are no available implementations of any of the updated algorithms for our target platform at the time of writing, so we are unable to confirm the applicability of our attacks experimentally. Further, in the interest of limiting the length of this paper, we do not provide detailed descriptions of how the individual algorithms work nor do we conduct an exhaustive evaluation of each of the algorithms, which we instead leave to future work.

12.1 FAEST

The second-round version of FAEST [14] introduced two main changes relevant for the presented side-channel and fault injection attacks. The first of these is an updated batch all-but-one vector commitment (BAVC), which uses a single large tree instead of τ individual trees as in the first-round variant. The second change relates to the proving of constraints in the QuickSilver proof, which is now performed using degree-3 constraints instead of degree-2 constraints, which entails further changes in how the witness bits and VOLE tags are used to construct the proof.

For the byte combine attack, only small changes would be required to adapt it to the second-round variant. In particular, because the use of degree-3 constraints changes the computation of the constraints in the proof of the encryption and thus eliminates the call to the `ByteCombine` procedure for the first round key, the attack would instead target the `StateToBytes` conversion of a later round key, corresponding to line 16 of `ZK.EncCstrnts` in the updated specification [14]. As shown in Section 7, the original cipher key can then be derived from the round key. The `StateToBytes` procedure uses the `ByteCombine` procedure internally, thus if the same implementation of this procedure is used, the rest of the attack can be applied without further changes.

The VOLE transpose attack and the VOLE conversion abort attacks are applicable without further changes, as the second-round variant still explicitly allows implementations to transpose the VOLE tag matrix and the `ConvertToVOLE` procedure is unchanged between the two versions. Obviously, for either attack to be successful against a given implementation, the implementation would need to use the same strategy for implementing these procedures as Degn et al. [25].

The counter increment skipping attack is not directly applicable, because the BAVC no longer requires deriving several root keys. However, as mentioned previously, it is possible to adapt the attack to focus on the generation of child nodes within a tree instead of the generation of root keys. The resulting attack, which we call the *extended counter increment skipping attack*, is then applicable to both the first-round variant of FAEST that uses multiple GGM trees and to BAVC-based schemes, such as the second-round variant of FAEST.

To understand how and when the attack works, we briefly describe the BAVC in the second-round variant of FAEST. Fig. 5 shows a simplified example of a BAVC with $L = 12$ leaves (in parameter set FAEST-128f, the commitment has $L = 3072$ leaves, but the structure of the tree remains the same). The idea is to derive a large GGM tree using a length-doubling PRG as before and to then assign the leaves of this tree to positions of the τ individual vectors in an interleaved manner, which reduces the number of nodes that need to be revealed compared to τ separate trees [15]. As the second-round variant of FAEST also employs grinding (a technique whereby certain values of Δ are considered invalid, which reduces the challenge space), not all vectors have the same length and the tree is therefore not balanced. The leaves are assigned to both large and small vectors starting from the right subtree (i.e. from the subtree starting at node 2 in Fig. 5), then from the left half of the left subtree (i.e. the subtree starting at

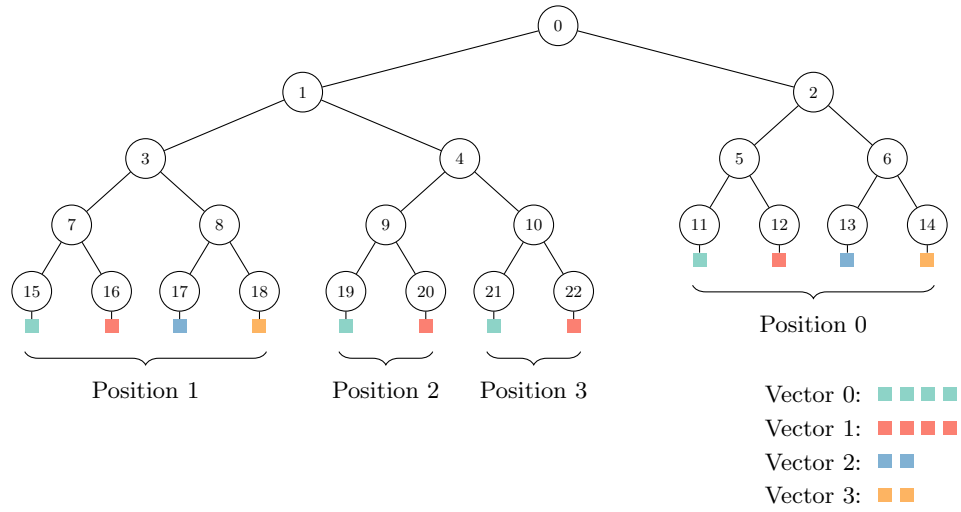


Fig. 5. Simplified illustration of the BAVC in the second-round variant of FAEST for $L = 12$ leaves. The commitment derives two vectors of size $2^k = 4$ and two vectors of size $2^{k-1} = 2$. Note that the leaves are interleaved (i.e. neighbouring leaves belong to different vectors), that the first position of each vector is derived from the leaves in the right subtree, and that in the right half of the left subtree the stride between positions changes from 4 to 2.

node 3), with the remaining positions of only the large vectors being assigned from the right half of the left subtree (i.e. the subtree starting at node 4).

The idea behind the extended counter increment skipping attack is to skip the incrementation of the counter prior to the derivation of the right child node for a specific node. As in the original counter increment skipping attack, this will cause the output to be the same for both nodes. If one of the nodes is subsequently revealed (because none of its children are missing) and the other node is not revealed (because at least one of its children is missing), it is possible to recompute the subtree with the missing value and thereby recover the secret key. Note that this attack only works if one of the nodes is explicitly revealed, which reduces its success probability. As the second-round variant of FAEST employs a per-node tweak to the PRG (meaning that the output of the PRG depends both on the value of the node and its position in the tree), it is not possible to create two identical subtrees and recover the missing value from the leaves, as before. This is not the case for the first-round variant with multiple GGM trees, where the attack will always succeed, as only one value is missing in each tree, which means that at least one of the children of each node will be revealed.

In the following, we describe how to estimate the probability of the attack to succeed against a BAVC when faulting certain nodes. Let

$$\text{Vec}_\alpha := \begin{cases} \{(\alpha - L + 1) \bmod \tau\} & \text{if } \alpha \in [L - 1..L - 1 + \tau 2^{k-1}) \\ \{(\alpha - L + 1 - \tau 2^{k-1}) \bmod \tau_1\} & \text{if } \alpha \in [L - 1 + \tau 2^{k-1}..2L - 1) \\ \text{Vec}_{2\alpha+1} \cup \text{Vec}_{2\alpha+2} & \text{otherwise} \end{cases}$$

be the set of vectors belonging to the leaves covered by the subtree starting at node α , and let

$$\text{Pos}_\alpha := \begin{cases} \{(\alpha - L + 1)/\tau\} & \text{if } \alpha \in [L - 1..L - 1 + \tau 2^{k-1}) \\ \{(\alpha - L + 1 - \tau 2^{k-1})/\tau_1 + 2^{k-1}\} & \text{if } \alpha \in [L - 1 + \tau 2^{k-1}..2L - 1) \\ \text{Pos}_{2\alpha+1} \cup \text{Pos}_{2\alpha+2} & \text{otherwise} \end{cases}$$

be the set of positions in these vectors. To simplify the notation, we define

$$v_\alpha^{(l)} := |\text{Vec}_\alpha \cap \{0, \dots, \tau_1 - 1\}|$$

$$v_\alpha^{(s)} := |\text{Vec}_\alpha \cap \{\tau_1, \dots, \tau - 1\}|$$

as the number of vectors belonging to the large and small VOLE instances (of size 2^k and 2^{k-1} respectively). Similarly, we define

$$p_\alpha^{(l)} := |\text{Pos}_\alpha|$$

$$p_\alpha^{(s)} := |\text{Pos}_\alpha \cap \{0, \dots, 2^{k-1} - 1\}|$$

as the number of positions belonging to the large and small VOLE instances.

The probability that exactly one of the children of a non-leaf node $\alpha \in [0..L-1]$ is revealed is given by

$$\begin{aligned} \Pr[\alpha] = & \left(\frac{2^k - p_{2\alpha+1}^{(l)}}{2^k} \right)^{v_{2\alpha+1}^{(l)}} \left(\frac{2^{k-1} - p_{2\alpha+1}^{(s)}}{2^{k-1}} \right)^{v_{2\alpha+1}^{(s)}} \\ & + \left(\frac{2^k - p_{2\alpha+2}^{(l)}}{2^k} \right)^{v_{2\alpha+2}^{(l)}} \left(\frac{2^{k-1} - p_{2\alpha+2}^{(s)}}{2^{k-1}} \right)^{v_{2\alpha+2}^{(s)}} \\ & - 2 \left(\frac{2^k - p_{\alpha}^{(l)}}{2^k} \right)^{v_{\alpha}^{(l)}} \left(\frac{2^{k-1} - p_{\alpha}^{(s)}}{2^{k-1}} \right)^{v_{\alpha}^{(s)}}. \end{aligned}$$

For the second-round parameter set **FAEST-128f**, this probability takes its maximum value of 0.516 for any of the nodes 11–18. In other words, when faulting the derivation of the right child of any of the nodes 11–18, the resulting tree will allow for the recovery of the secret key with probability $0.516 \cdot \Pr[\text{fault}]$, which may be sufficient to conduct single-execution attacks, provided that the probability of a successful fault, $\Pr[\text{fault}]$, is high enough.

Note that the rejection sampling during the opening of the vector commitment caused by the grinding in the second-round variant only affects the attack if the vector commitment is recomputed during the opening (as is the case in the implementation by Degn et al. [25] for the first-round variant), in which case it would be necessary to inject a fault into an opening that is not rejected. Due to the computational costs associated with this approach, it seems unlikely that implementations of the second-round variant would choose such a strategy, however.

12.2 SDitH

Among the remaining four schemes, the second-round variant of SDitH [2] is the only other scheme that explicitly uses the VOLEitH paradigm and is thus most closely related to FAEST.

As previously pointed out, whether an attack is applicable to a given implementation depends on how the implementation realises certain functionality. Nonetheless, we sketch analogous attacks on SDitH for each of the presented attacks on FAEST, based on the current specification. Future work should consider these attacks in the context of a concrete implementation.

As an analogue to the byte combine attack, there exists the **SampleFieldElements** procedure (see [2]), which converts output bytes from a PRG into vectors of field elements. For the field $\mathbb{F}_2$, it thus performs the inverse of the **ByteCombine** procedure in FAEST. When used in line 7 of the **BLC.Commit** procedure, it may leak information about the PRG output, which can be used to compute the masking vector $\mathbf{u}$ and from it the witness, which reveals the secret key of the scheme, similar to the presented attack.

For the VOLE transpose attack, there exists a function ψ that performs a conversion $\mathbb{F}_2^k \rightarrow \mathbb{F}_2^\lambda$ using bitwise multiplication. This function is applied to each row of the VOLE tag matrix $\mathbf{V}$ in line 19 of `BLC.Commit` and may thus leak information about the VOLE tags, which can be used to recover the secret key, similar to the presented attack.

For the VOLE conversion abort attack, it may be possible to abort or skip the loop in lines 5 to 9 of the `BLC.Commit` procedure, thus causing the masking vector $\mathbf{u}$ to be zero-initialised or to be dependent only on the first seed, which can be used to recover the secret key of the scheme, similar to the presented attack.

Finally, the extended counter increment skipping attack may also be applicable to SDitH, as the underlying vector commitment is similar (though the expansion of the child nodes is realised as two separate encryptions with a manually incremented counter, which is likely still vulnerable).

12.3 Schemes in the Polynomial Interactive Oracle Proof formalism

The remaining three schemes, Mirath [1], RYDE [6], and MQOM [18] are instead based on the TCitH with GGM trees paradigm [29], but are stated in terms of the Polynomial Interactive Oracle Proof (PIOP) formalism [28]. This formalism unifies both TCitH and VOLEitH by operating on polynomials (instead of Shamir secret sharings as in TCitH or VOLE correlations as in VOLEitH) and the schemes are thus closely related to VOLEitH and can be instantiated using either paradigm, which allows some of the attacks presented in this paper to be applied to them.

However, as these schemes operate on polynomials and by default use TCitH with GGM trees, the byte combine and VOLE transpose attacks, which operate on witness bits or VOLE tags, are not directly applicable. This should not be misinterpreted to mean that such attacks are impossible, but instead suggests that any attacks are likely to more closely resemble the attack presented by Godard et al. [32], which targets the evaluation of a polynomial in the TCitH version of SDitH. We therefore leave a closer analysis of these types of attacks to future work.

Mirath For Mirath [1], as an analogue to the VOLE conversion abort attack, it may be possible to abort or skip the loop in lines 5 to 8 of the `CommitWitnessPolynomials` procedure (see [1]), thus causing the $\mathbf{S}_{\text{acc}}$ and $\mathbf{C}'_{\text{acc}}$ values to be zero-initialised or to be dependent only on the first seed, which reveals the matrices $\mathbf{S}$ and $\mathbf{C}'$, which are an equivalent secret key for the scheme.

Additionally, the extended counter increment skipping attack may also be applicable to Mirath, as the underlying vector commitment is similar. As in SDitH, the expansion of the child nodes is realised as two separate encryptions. However, in Mirath these encryptions are performed on two distinct messages and thus do not involve a counter. The messages differ only in the last bit so it is nonetheless likely that implementations would realise this functionality in a similar way.

RYDE The attacks on RYDE [6] are identical to those on Mirath. As an analogue to the VOLE conversion abort attack, it may be possible to abort or skip the loop in lines 9 to 14 of the `RYDE.Sign` procedure (see [6]), thus causing the $\text{acc}_{s'}$ and acc_C values to be zero-initialised or to be dependent only on the first seed, which reveals the matrices s' and C , which are an equivalent secret key for the scheme.

The extended counter increment skipping attack may also be applicable, though the same stipulation regarding the absence of a counter applies here. As both messages that are being encrypted again differ only in the last bit, it is likely that implementations would realise this functionality in a similar way.

MQOM The last scheme, MQOM [18], is structurally the most different from the other schemes, as it does not use a BAVC. Instead, MQOM uses a variant of the half-tree technique [34,24,20] called the correlated half-tree technique [36,43]. In a half-tree, the child nodes $k_{2\alpha+1}, k_{2\alpha+2}$ of a node k_α are computed as $k_{2\alpha+1} := \text{PRG}(k_\alpha)$ and $k_{2\alpha+2} := k_{2\alpha+1} \oplus k_\alpha$, i.e. with a single evaluation of the PRG. In a correlated tree, the XOR of all nodes at a specific level is the same across all levels of the tree, which further allows minimising communication by omitting certain values. For more details, we refer to the specification [18].

An important insight is that the half-tree technique prevents the extended counter increment skipping attack, as the evaluation of the PRG is only performed once. However, it may be possible to adapt the attack to skip the XOR during the computation of the second child $k_{2\alpha+2} := k_{2\alpha+1} \oplus k_\alpha$. Depending on the implementation, this may cause $k_{2\alpha+2} = k_{2\alpha+1}$, which is equivalent to the extended counter increment skipping attack, or it may cause $k_{2\alpha+2} = k_\alpha$, which allows recomputing $k_{2\alpha+1}$ if $k_{2\alpha+2}$ is revealed. As a countermeasure against this attack, it may be possible to ensure that values $k_\alpha, k_{2\alpha+1}$ and $k_{2\alpha+2}$ are stored in different registers, in which case skipping the XOR would cause $k_{2\alpha+2}$ to be uninitialised instead, which may prevent the attack.

The half-tree technique therefore appears to be a promising approach for protecting against fault attacks in the all-but-one vector commitment and future work should consider its applicability in BAVCs and its resistance to fault attacks more closely.

12.4 Schemes with SHAKE-based GGM trees

Among the remaining submissions in the competition for additional signature schemes are three schemes (PERK, CROSS, and LESS) that use GGM trees with SHAKE as the PRG. As explained in Section 11.5, this prevents the counter increment skipping attack (and also its extended variant). However, it has been shown [39,38,58] that it is possible to control the output of SHAKE via a single fault injection. It may therefore be possible to adapt the attack to these algorithms.

PERK For PERK [7], it may be possible to fix one or more seeds $\theta^{(e)}$ in line 4 of the PERK sign algorithm (see [7]), which are used as roots for the GGM trees, to a known value using the mentioned techniques. A tree derived from a known root would allow recomputing its leaves $\theta_i^{(e)}$, and thus its party’s share of the permutation $\pi_i^{(e)}$, which can be used to recover the permutation π , which is an equivalent secret key of the scheme.

LESS For LESS [13] a similar attack to the extended counter increment skipping attack may be possible by fixing the value of the child nodes in line 9 of the BuildGGM procedure (see [13]). This would then allow deriving the $\text{seed}^{(i)}$ values and from these the ephemeral monomial maps $\tilde{\mu}^{(i)}$, which may allow recomputing the monomial maps τ_i from the coset representatives, which are an equivalent secret key for the scheme.

Note that this attack is different from the attack of Mondal et al. [45], as it allows recomputing (a subtree of) the GGM tree directly, instead of tricking the algorithm into revealing parts of these values.

CROSS The attack on CROSS [12] is identical. Through knowledge of the subtree it may be possible to reveal values $\bar{e}_G'[i]$ in line 4 of the CROSS sign algorithm (see [12]), which allows recomputing $\bar{e}_G$ from $\bar{v}_G[i]$ and thus also allows recomputing e , which is an equivalent secret key for the scheme.

In practice, such an attack may be difficult, due to the proposed parallelisation of the tree expansion introduced in the second-round variant. This optimised version derives several children simultaneously, which may prevent the mentioned techniques from being applied or reduce their success probability.

13 Conclusion

This paper presented the first evaluation of the resistance of VOLEitH-based signature schemes to side-channel and fault injection attacks. We demonstrated how knowledge of individual components of the underlying VOLE correlations can be exploited to recover the secret key. We further presented practical single-trace deep learning-assisted power-based side channel attacks and single-execution first-order voltage fault injection attacks on a masked implementation of FAEST, successfully recovering the full secret key with a probability greater than 87%. We also proposed countermeasures against these attacks.

Our results highlight the importance of protecting operations involving witness bits and VOLE tags from side-channel attacks, as well as the need to ensure that operations involving the computation of VOLE and vector commitments are resilient to fault injection attacks.

Acknowledgements

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation and by the Swedish Civil Contingencies Agency (Grant No. 2020-11632).

References

1. Adj, G., Aragon, N., Barbero, S., Bardet, M., Bellini, E., Bidoux, L., Chi-Domínguez, J.J., Dyseryn, V., Esser, A., Feneuil, T., Gaborit, P., Neveu, R., Rivain, M., Rivera-Zamarripa, L., Sanna, C., Tillich, J.P., Verbel, J., Zweydinger, F.: Mirath signature scheme. <https://pqc-mirath.org/assets/downloads/mirath-v2.0.pdf> (February 2025)
2. Aguilar Melchor, C., Bettaieb, S., Bidoux, L., Feneuil, T., Gaborit, P., Gama, N., Gueron, S., Howe, J., Hülsing, A., Joseph, D., Joux, A., Kulkarni, M., Persichetti, E., H. Randrianarisoa, T., Rivain, M., Yue, D.: The syndrome decoding in the head (SD-in-the-Head) signature scheme. <https://sdith.org/docs/sdith-v2.0.pdf> (February 2025)
3. Aguilar Melchor, C., Feneuil, T., Gama, N., Gueron, S., Howe, J., Joseph, D., Joux, A., Persichetti, E., H. Randrianarisoa, T., Rivain, M., Yue, D.: The syndrome decoding in the head (SD-in-the-Head) signature scheme. <https://sdith.org/docs/sdith-package-v1.zip> (May 2023)
4. Albrecht, M.R., Rechberger, C., Schneider, T., Tiessen, T., Zohner, M.: Ciphers for MPC and FHE. In: Oswald, E., Fischlin, M. (eds.) *Advances in Cryptology - Proceedings of EUROCRYPT'2015*. Lecture Notes in Computer Science, vol. 9056, pp. 430–454. Springer (2015). https://doi.org/10.1007/978-3-662-46800-5_17
5. Amiet, D., Curiger, A., Leuenberger, L., Zbinden, P.: Defeating NewHope with a single trace. In: Ding, J., Tillich, J. (eds.) *Proceedings of Post-Quantum Cryptography - 11th International Conference, PQCrypto'2020*. Lecture Notes in Computer Science, vol. 12100, pp. 189–205. Springer (2020). https://doi.org/10.1007/978-3-030-44223-1_11
6. Aragon, N., Bardet, M., Bidoux, L., Chi-Domínguez, J.J., Dyseryn, V., Feneuil, T., Gaborit, P., Joux, A., Neveu, R., Rivain, M., Tillich, J.P., Vinçotte, A.: RYDE signature scheme. https://www.pqc-ryde.org/assets/downloads/ryde_specification_v2.0.pdf (February 2025)
7. Araj, N., Bettaieb, S., Bidoux, L., Budroni, A., Dyseryn, V., Esser, A., Feneuil, T., Gaborit, P., Kulkarni, M., Mateu, V., Palumbi, M., Perin, L., Rivain, M., Tillich, J.P., Xagawa, K.: PERK. <https://pqc-perk.org/assets/downloads/perk-v2.0.pdf> (February 2025)
8. Aranha, D.F., Berndt, S., Eisenbarth, T., Seker, O., Takahashi, A., Wilke, L., Zaverucha, G.: Side-channel protections for Picnic signatures. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2021**(4), 239–282 (2021). <https://doi.org/10.46586/TCHES.V2021.I4.239-282>
9. Aulbach, T., Campos, F., Krämer, J., Samardjiska, S., Stöttinger, M.: Separating oil and vinegar with a single trace side-channel assisted Kipnis-Shamir attack on UOV. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2023**(3), 221–245 (2023). <https://doi.org/10.46586/TCHES.V2023.I3.221-245>

10. Aulbach, T., Marzougui, S., Seifert, J., Ulitzsch, V.Q.: MAYo or MAY-not: Exploring implementation security of the post-quantum signature scheme MAYO against physical attacks. In: Workshop on Fault Detection and Tolerance in Cryptography, FDTC'2024. pp. 28–33. IEEE (2024). <https://doi.org/10.1109/FDTC64268.2024.00012>
11. Backlund, L., Ngo, K., Gärtner, J., Dubrova, E.: Secret key recovery attack on masked and shuffled implementations of crystals-kyber and saber. In: Zhou, J., Batina, L., Li, Z., Lin, J., Losiouk, E., Majumdar, S., Mashima, D., Meng, W., Picek, S., Rahman, M.A., Shao, J., Shimaoka, M., Soremekun, E.O., Su, C., Teh, J.S., Udovenko, A., Wang, C., Zhang, L.Y., Zhauniarovich, Y. (eds.) Proceedings of the Applied Cryptography and Network Security Workshops - ACNS'2023. Lecture Notes in Computer Science, vol. 13907, pp. 159–177. Springer (2023). https://doi.org/10.1007/978-3-031-41181-6_9
12. Baldi, M., Barengi, A., Battagliola, M., Bitzer, S., Gianvecchio, M., Karl, P., Manganiello, F., Pavoni, A., Pelosi, G., Santini, P., Schupp, J., Signorini, E., Slaughter, F., Wachter-Zeh, A., Weger, V.: CROSS. https://www.cross-crypto.com/CROSS_Specification_v2.1.pdf (March 2025)
13. Baldi, M., Barengi, A., Beckwith, L., Biasse, J.F., Chou, T., Esser, A., Gaj, K., Karl, P., Mohajerani, K., Pelosi, G., Persichetti, E., Saarinen, M.J.O., Santini, P., Wallace, R., Zweydinger, F.: LESS: Linear equivalence signature scheme. <https://www.less-project.com/LESS-2025-02-07.pdf> (February 2025)
14. Baum, C., Beullens, W., Braun, L., de Saint Guilhem, C.D., Klooß, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: FAEST v2: Algorithm Specifications. <https://faest.info/faest-spec-v2.0.pdf> (February 2025)
15. Baum, C., Beullens, W., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: One tree to rule them all: Optimizing GGM trees and OWFs for post-quantum signatures. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - Proceedings of ASIACRYPT'2024. Lecture Notes in Computer Science, vol. 15484, pp. 463–493. Springer (2024). https://doi.org/10.1007/978-981-96-0875-1_15
16. Baum, C., Braun, L., de Saint Guilhem, C.D., Klooß, M., Orsini, E., Roy, L., Scholl, P.: Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-Head. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology - Proceedings of CRYPTO'2023. Lecture Notes in Computer Science, vol. 14085, pp. 581–615. Springer (2023). https://doi.org/10.1007/978-3-031-38554-4_19
17. Baum, C., Braun, L., de Saint Guilhem, C.D., Klooß, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: FAEST: Algorithm Specifications. <https://faest.info/faest-spec-v1.1.pdf> (July 2023)
18. Benadjila, R., Bouillaguet, C., Feneuil, T., Rivain, M.: MQOM: MQ on my mind. <https://mqom.org/docs/mqom-v2.0.pdf> (February 2025)
19. Beullens, W., Campos, F., Celi, S., Hess, B., Kannwischer, M.J.: MAYO. <https://pqmayo.org/assets/specs/mayo.pdf> (June 2023)
20. Bui, D., Cong, K., de Saint Guilhem, C.D.: Improved all-but-one vector commitment with applications to post-quantum signatures. IACR Cryptol. ePrint Arch. p. 97 (2024), <https://eprint.iacr.org/2024/097>
21. Chari, S., Jutla, C.S., Rao, J.R., Rohatgi, P.: Towards sound approaches to counteract power-analysis attacks. In: Wiener, M.J. (ed.) Advances in Cryptology - Proceedings of CRYPTO'99. Lecture Notes in Computer Science, vol. 1666, pp. 398–412. Springer (1999). https://doi.org/10.1007/3-540-48405-1_26

22. Chase, M., Derler, D., Goldfeder, S., Orlandi, C., Ramacher, S., Rechberger, C., Slamanig, D., Zaverucha, G.: Post-quantum zero-knowledge and signatures from symmetric-key primitives. In: Thuraisingham, B., Evans, D., Malkin, T., Xu, D. (eds.) *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS'2017*. pp. 1825–1842. ACM (2017). <https://doi.org/10.1145/3133956.3133997>
23. Chen, Z., Karabulut, E., Aysu, A., Ma, Y., Jing, J.: An efficient non-profiled side-channel attack on the CRYSTALS-Dilithium post-quantum signature. In: *Proceedings of the 39th IEEE International Conference on Computer Design, ICCD'2021*. pp. 583–590. IEEE (2021). <https://doi.org/10.1109/ICCD53106.2021.00094>
24. Cui, H., Liu, H., Yan, D., Yang, K., Yu, Y., Zhang, K.: ReSolveD: Shorter signatures from regular syndrome decoding and VOLE-in-the-Head. In: Tang, Q., Teague, V. (eds.) *Proceedings of Public-Key Cryptography – PKC 2024*. pp. 229–258. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-57718-5_8
25. Degn, J.T., Eilath, J., Nielsen, K.: pqm4-faest. <https://github.com/johandegn/pqm4-faest>
26. Dobraunig, C., Kales, D., Rechberger, C., Schafneggler, M., Zaverucha, G.: Shorter signatures based on tailor-made minimalist symmetric-key crypto. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS'2022*. pp. 843–857. ACM (2022). <https://doi.org/10.1145/3548606.3559353>
27. Dubrova, E., Ngo, K., Gärtner, J., Wang, R.: Breaking a fifth-order masked implementation of CRYSTALS-Kyber by copy-paste. In: Fukumitsu, M., Hasegawa, S. (eds.) *Proceedings of the 10th ACM Asia Public-Key Cryptography Workshop, APKC 2023*. pp. 10–20. ACM (2023). <https://doi.org/10.1145/3591866.3593072>
28. Feneuil, T.: The Polynomial-IOP vision of the latest MPCitH frameworks for signature schemes. *Post-Quantum Cryptography Trimester - Second Workshop*, Institut Henri Poincaré, Paris, France (November 2024), https://www.thibault-feneuil.fr/docs/talks/2024-11-08_IHP-PQC-piop-based-sigs.pdf
29. Feneuil, T., Rivain, M.: Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments. *Cryptology ePrint Archive*, Paper 2023/1573 (2023), <https://eprint.iacr.org/2023/1573>
30. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) *Advances in Cryptology - Proceedings of CRYPTO'86. Lecture Notes in Computer Science*, vol. 263, pp. 186–194. Springer (1986). https://doi.org/10.1007/3-540-47721-7_12
31. Gellersen, T., Seker, O., Eisenbarth, T.: Differential power analysis of the Picnic signature scheme. In: Cheon, J.H., Tillich, J. (eds.) *Proceedings of Post-Quantum Cryptography - 12th International Workshop, PQCrypto'2021. Lecture Notes in Computer Science*, vol. 12841, pp. 177–194. Springer (2021). https://doi.org/10.1007/978-3-030-81293-5_10
32. Godard, J., Aragon, N., Gaborit, P., Loiseau, A., Maillard, J.: Single trace side-channel attack on the MPC-in-the-head framework. *Cryptology ePrint Archive*, Paper 2024/1882 (2024), <https://eprint.iacr.org/2024/1882>
33. Goldreich, O., Goldwasser, S., Micali, S.: How to construct random functions (extended abstract). In: *Proceedings of the 25th Annual Symposium on Foundations of Computer Science*. pp. 464–479. IEEE Computer Society (1984). <https://doi.org/10.1109/SFCS.1984.715949>

34. Guo, X., Yang, K., Wang, X., Zhang, W., Xie, X., Zhang, J., Liu, Z.: Half-tree: Halving the cost of tree expansion in COT and DPF. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology - Proceedings of EUROCRYPT'2023*. Lecture Notes in Computer Science, vol. 14004, pp. 330–362. Springer (2023). https://doi.org/10.1007/978-3-031-30545-0_12
35. Hou, X., Breier, J.: *Cryptography and Embedded Systems Security*. Springer Nature Switzerland, Cham, 1st edn. (2024). <https://doi.org/10.1007/978-3-031-62205-2>
36. Huth, J., Joux, A.: MPC in the head using the subfield bilinear collision problem. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology - Proceedings of CRYPTO'2024*. Lecture Notes in Computer Science, vol. 14920, pp. 39–70. Springer (2024). https://doi.org/10.1007/978-3-031-68376-3_2, https://doi.org/10.1007/978-3-031-68376-3_2
37. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-knowledge proofs from secure multiparty computation. *SIAM J. Comput.* **39**(3), 1121–1152 (2009). <https://doi.org/10.1137/080725398>
38. Jendral, S., Dubrova, E.: MAYO key recovery by fixing vinegar seeds. *IACR Commun. Cryptol.* **1**(4), 17 (2024). <https://doi.org/10.62056/AB0LJBKRZ>
39. Jendral, S., Mattsson, J.P., Dubrova, E.: A single-trace fault injection attack on hedged module lattice digital signature algorithm (ML-DSA). In: *Workshop on Fault Detection and Tolerance in Cryptography, FDTC'2024*. pp. 34–43. IEEE (2024). <https://doi.org/10.1109/FDTC64268.2024.00013>
40. Jendral, S., Ngo, K., Wang, R., Dubrova, E.: Breaking SCA-Protected CRYSTALS-Kyber with a single trace. In: *IEEE International Symposium on Hardware Oriented Security and Trust, HOST'2024*. pp. 70–73. IEEE (2024). <https://doi.org/10.1109/HOST55342.2024.10545390>
41. Jendral, S., Dubrova, E.: Single-trace side-channel attacks on MAYO exploiting leaky modular multiplication. *Cryptology ePrint Archive*, Paper 2024/1850 (2024), <https://eprint.iacr.org/2024/1850>
42. Karabulut, E., Aysu, A.: FALCON down: Breaking FALCON post-quantum signature scheme through side-channel attacks. In: *Proceedings of the 58th ACM/IEEE Design Automation Conference, DAC'2021*. pp. 691–696. IEEE (2021). <https://doi.org/10.1109/DAC18074.2021.9586131>
43. Kim, S., Lee, B., Son, M.: Relaxed vector commitment for shorter signatures. *Cryptology ePrint Archive*, Paper 2024/1004 (2024), <https://eprint.iacr.org/2024/1004>
44. Krahmer, E., Pessl, P., Land, G., Güneysu, T.: Correction fault attacks on randomized CRYSTALS-Dilithium. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2024**(3), 174–199 (2024). <https://doi.org/10.46586/TCHES.V2024.I3.174-199>
45. Mondal, P., Adhikary, S., Kundu, S., Karmakar, A.: ZKFault: Fault attack analysis on zero-knowledge based post-quantum digital signature schemes. In: Chung, K., Sasaki, Y. (eds.) *Advances in Cryptology - Proceedings of ASIACRYPT'2024*. Lecture Notes in Computer Science, vol. 15491, pp. 132–167. Springer (2024). https://doi.org/10.1007/978-981-96-0944-4_5
46. National Institute of Standards and Technology: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. Tech. Rep. NIST FIPS 202, National Institute of Standards and Technology, Gaithersburg, MD (August 2015). <https://doi.org/10.6028/NIST.FIPS.202>
47. National Institute of Standards and Technology: NIST announces additional digital signature candidates for the PQC standardization process. <https://csrc.nist.gov/news/2023/additional-pqc-digital-signature-candidates> (June 2023)

48. National Institute of Standards and Technology: Module-Lattice-Based Digital Signature Standard. Tech. Rep. NIST FIPS 204, National Institute of Standards and Technology, Gaithersburg, MD (August 2024). <https://doi.org/10.6028/NIST.FIPS.204>
49. National Institute of Standards and Technology: Module-Lattice-Based Key Encapsulation Mechanism Standard. Tech. Rep. NIST FIPS 203, National Institute of Standards and Technology, Gaithersburg, MD (August 2024). <https://doi.org/10.6028/NIST.FIPS.203>
50. National Institute of Standards and Technology: NIST announces 14 candidates to advance to the second round of the additional digital signatures for the post-quantum cryptography standardization process. <https://csrc.nist.gov/news/2024/pqc-digital-signature-second-round-announcement> (October 2024)
51. National Institute of Standards and Technology: Stateless Hash-Based Digital Signature Standard. Tech. Rep. NIST FIPS 205, National Institute of Standards and Technology, Gaithersburg, MD (August 2024). <https://doi.org/10.6028/NIST.FIPS.205>
52. Ngo, K., Dubrova, E., Johansson, T.: Breaking masked and shuffled CCA secure Saber KEM by power analysis. In: Chang, C., Rührmair, U., Katzenbeisser, S., Mukhopadhyay, D. (eds.) ASHES@CCS 2021: Proceedings of the 5th Workshop on Attacks and Solutions in Hardware Security. pp. 51–61. ACM (2021). <https://doi.org/10.1145/3474376.3487277>
53. Roy, L.: SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the Minicrypt model. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology - Proceedings of CRYPTO’2022. Lecture Notes in Computer Science, vol. 13507, pp. 657–687. Springer (2022). https://doi.org/10.1007/978-3-031-15802-5_23
54. Seker, O., Berndt, S., Wilke, L., Eisenbarth, T.: SNI-in-the-head: Protecting MPC-in-the-head protocols against side-channel analysis. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, CCS’20. pp. 1033–1049. ACM (2020). <https://doi.org/10.1145/3372297.3417889>
55. Sim, B., Kwon, J., Lee, J., Kim, I., Lee, T., Han, J., Yoon, H.J., Cho, J., Han, D.: Single-trace attacks on message encoding in lattice-based KEMs. IEEE Access **8**, 183175–183191 (2020). <https://doi.org/10.1109/ACCESS.2020.3029521>
56. STMicroelectronics: https://www.st.com/resource/en/reference_manual/rm0090-stm32f405415-stm32f407417-stm32f427437-and-stm32f429439-advanced-armbased-32bit-mcus-stmicroelectronics.pdf (June 2024)
57. Wang, R., Dubrova, E.: A shared key recovery attack on a masked implementation of CRYSTALS-Kyber’s encapsulation algorithm. In: Mosbah, M., Sèdes, F., Tawbi, N., Ahmed, T., Boulahia-Cuppens, N., García-Alfaro, J. (eds.) Proceedings of Foundations and Practice of Security - 16th International Symposium, FPS 2023. Lecture Notes in Computer Science, vol. 14551, pp. 424–439. Springer (2023). https://doi.org/10.1007/978-3-031-57537-2_26
58. Wang, Y., Yu, J., Qu, S., Zhang, X., Li, X., Zhang, C., Gu, D.: Mind the faulty Keccak: A practical fault injection attack scheme apply to all phases of ML-KEM and ML-DSA. Cryptology ePrint Archive, Paper 2024/1522 (2024), <https://eprint.iacr.org/2024/1522>
59. Yang, K., Sarkar, P., Weng, C., Wang, X.: Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In: Kim, Y., Kim, J., Vigna, G., Shi, E. (eds.) Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS’21. pp. 2986–3001. ACM (2021). <https://doi.org/10.1145/3460120.3484556>